Colección III de Proyectos de Curso
Ingeniería de Software

Guía detallada de formulación, desarrollo, arquitectura, pruebas y evaluación

Propósito

Este documento propone proyectos integradores en los que la inteligencia artificial no reemplaza la Ingeniería de Software: se incorpora como un componente dentro de un sistema completo con requisitos, arquitectura, persistencia, API, interfaz, pruebas, seguridad, despliegue, métricas y documentación.

Docente: Julián René Muñoz Burbano
2026

Índice

1. Orientación general del curso **7**
1.1. Arquitectura de referencia . . . 7
1.2. Proceso recomendado . . . 7

2. Catálogo resumido **7**

3. Proyecto 1: Sistema inteligente de gestión de filas y tiempos de espera **9**
3.1. Descripción y objetivo . . . 9
3.2. Usuarios, alcance y requisitos sugeridos . . . 9
3.3. Arquitectura propuesta . . . 9
3.4. Tecnologías sugeridas . . . 9
3.5. Paso a paso de implementación . . . 10
3.6. Diagrama del flujo de extremo a extremo . . . 11
3.7. Plan de pruebas . . . 11
3.8. Entregables específicos . . . 11
3.9. Criterios de aceptación académica . . . 11

4. Proyecto 2: Plataforma inteligente de gestión de donaciones y distribución de ayudas **12**
4.1. Descripción y objetivo . . . 12
4.2. Usuarios, alcance y requisitos sugeridos . . . 12
4.3. Arquitectura propuesta . . . 12
4.4. Tecnologías sugeridas . . . 12
4.5. Paso a paso de implementación . . . 13
4.6. Diagrama del flujo de extremo a extremo . . . 14
4.7. Plan de pruebas . . . 14
4.8. Entregables específicos . . . 14
4.9. Criterios de aceptación académica . . . 14

5. Proyecto 3: Sistema de monitoreo inteligente de redes informáticas **15**
5.1. Descripción y objetivo . . . 15
5.2. Usuarios, alcance y requisitos sugeridos . . . 15
5.3. Arquitectura propuesta . . . 15
5.4. Tecnologías sugeridas . . . 15
5.5. Paso a paso de implementación . . . 16
5.6. Diagrama del flujo de extremo a extremo . . . 17
5.7. Plan de pruebas . . . 17
5.8. Entregables específicos . . . 17
5.9. Criterios de aceptación académica . . . 17

6. Proyecto 4: Plataforma de gestión de restaurantes con predicción de demanda **18**
6.1. Descripción y objetivo . . . 18
6.2. Usuarios, alcance y requisitos sugeridos . . . 18
6.3. Arquitectura propuesta . . . 18
6.4. Tecnologías sugeridas . . . 18
6.5. Paso a paso de implementación . . . 19
6.6. Diagrama del flujo de extremo a extremo . . . 20
6.7. Plan de pruebas . . . 20

6.8. Entregables específicos . . . 20
6.9. Criterios de aceptación académica . . . 20

7. Proyecto 5: Sistema inteligente para gestión de préstamos de equipos universitarios 21
7.1. Descripción y objetivo . . . 21
7.2. Usuarios, alcance y requisitos sugeridos . . . 21
7.3. Arquitectura propuesta . . . 21
7.4. Tecnologías sugeridas . . . 21
7.5. Paso a paso de implementación . . . 22
7.6. Diagrama del flujo de extremo a extremo . . . 23
7.7. Plan de pruebas . . . 23
7.8. Entregables específicos . . . 23
7.9. Criterios de aceptación académica . . . 23

8. Proyecto 6: Plataforma inteligente de análisis de reuniones 24
8.1. Descripción y objetivo . . . 24
8.2. Usuarios, alcance y requisitos sugeridos . . . 24
8.3. Arquitectura propuesta . . . 24
8.4. Tecnologías sugeridas . . . 24
8.5. Paso a paso de implementación . . . 25
8.6. Diagrama del flujo de extremo a extremo . . . 26
8.7. Plan de pruebas . . . 26
8.8. Entregables específicos . . . 26
8.9. Criterios de aceptación académica . . . 26

9. Proyecto 7: Sistema de gestión de mantenimiento de software con priorización de deuda técnica 27
9.1. Descripción y objetivo . . . 27
9.2. Usuarios, alcance y requisitos sugeridos . . . 27
9.3. Arquitectura propuesta . . . 27
9.4. Tecnologías sugeridas . . . 27
9.5. Paso a paso de implementación . . . 28
9.6. Diagrama del flujo de extremo a extremo . . . 29
9.7. Plan de pruebas . . . 29
9.8. Entregables específicos . . . 29
9.9. Criterios de aceptación académica . . . 29

10. Proyecto 8: Plataforma inteligente para análisis de experiencia del cliente omnicanal 30
10.1. Descripción y objetivo . . . 30
10.2. Usuarios, alcance y requisitos sugeridos . . . 30
10.3. Arquitectura propuesta . . . 30
10.4. Tecnologías sugeridas . . . 30
10.5. Paso a paso de implementación . . . 31
10.6. Diagrama del flujo de extremo a extremo . . . 32
10.7. Plan de pruebas . . . 32
10.8. Entregables específicos . . . 32
10.9. Criterios de aceptación académica . . . 32

11. Proyecto 9: Sistema de monitoreo de cultivos con predicción de necesidades de riego 33
11.1. Descripción y objetivo . . . 33
11.2. Usuarios, alcance y requisitos sugeridos . . . 33
11.3. Arquitectura propuesta . . . 33
11.4. Tecnologías sugeridas . . . 33
11.5. Paso a paso de implementación . . . 34
11.6. Diagrama del flujo de extremo a extremo . . . 35
11.7. Plan de pruebas . . . 35
11.8. Entregables específicos . . . 35
11.9. Criterios de aceptación académica . . . 35

12. Proyecto 10: Plataforma de gestión de gimnasios con recomendación de actividades 36
12.1. Descripción y objetivo . . . 36
12.2. Usuarios, alcance y requisitos sugeridos . . . 36
12.3. Arquitectura propuesta . . . 36
12.4. Tecnologías sugeridas . . . 36
12.5. Paso a paso de implementación . . . 37
12.6. Diagrama del flujo de extremo a extremo . . . 38
12.7. Plan de pruebas . . . 38
12.8. Entregables específicos . . . 38
12.9. Criterios de aceptación académica . . . 38

13. Proyecto 11: Sistema inteligente de análisis de documentos de soporte técnico 39
13.1. Descripción y objetivo . . . 39
13.2. Usuarios, alcance y requisitos sugeridos . . . 39
13.3. Arquitectura propuesta . . . 39
13.4. Tecnologías sugeridas . . . 39
13.5. Paso a paso de implementación . . . 40
13.6. Diagrama del flujo de extremo a extremo . . . 41
13.7. Plan de pruebas . . . 41
13.8. Entregables específicos . . . 41
13.9. Criterios de aceptación académica . . . 41

14. Proyecto 12: Plataforma de administración de condominios con clasificación de solicitudes 42
14.1. Descripción y objetivo . . . 42
14.2. Usuarios, alcance y requisitos sugeridos . . . 42
14.3. Arquitectura propuesta . . . 42
14.4. Tecnologías sugeridas . . . 42
14.5. Paso a paso de implementación . . . 43
14.6. Diagrama del flujo de extremo a extremo . . . 44
14.7. Plan de pruebas . . . 44
14.8. Entregables específicos . . . 44
14.9. Criterios de aceptación académica . . . 44

15. Proyecto 13: Sistema inteligente de control de cadena de frío 45
15.1. Descripción y objetivo . . . 45
15.2. Usuarios, alcance y requisitos sugeridos . . . 45
15.3. Arquitectura propuesta . . . 45
15.4. Tecnologías sugeridas . . . 45

15.5. Paso a paso de implementación . . . 46
15.6. Diagrama del flujo de extremo a extremo . . . 47
15.7. Plan de pruebas . . . 47
15.8. Entregables específicos . . . 47
15.9. Criterios de aceptación académica . . . 47

16. Proyecto 14: Plataforma de análisis de tráfico vehicular mediante visión artificial 48
16.1. Descripción y objetivo . . . 48
16.2. Usuarios, alcance y requisitos sugeridos . . . 48
16.3. Arquitectura propuesta . . . 48
16.4. Tecnologías sugeridas . . . 48
16.5. Paso a paso de implementación . . . 49
16.6. Diagrama del flujo de extremo a extremo . . . 50
16.7. Plan de pruebas . . . 50
16.8. Entregables específicos . . . 50
16.9. Criterios de aceptación académica . . . 50

17. Proyecto 15: Sistema inteligente de clasificación de correos corporativos 51
17.1. Descripción y objetivo . . . 51
17.2. Usuarios, alcance y requisitos sugeridos . . . 51
17.3. Arquitectura propuesta . . . 51
17.4. Tecnologías sugeridas . . . 51
17.5. Paso a paso de implementación . . . 52
17.6. Diagrama del flujo de extremo a extremo . . . 53
17.7. Plan de pruebas . . . 53
17.8. Entregables específicos . . . 53
17.9. Criterios de aceptación académica . . . 53

18. Proyecto 16: Plataforma de gestión de laboratorios con predicción de utilización 54
18.1. Descripción y objetivo . . . 54
18.2. Usuarios, alcance y requisitos sugeridos . . . 54
18.3. Arquitectura propuesta . . . 54
18.4. Tecnologías sugeridas . . . 54
18.5. Paso a paso de implementación . . . 55
18.6. Diagrama del flujo de extremo a extremo . . . 56
18.7. Plan de pruebas . . . 56
18.8. Entregables específicos . . . 56
18.9. Criterios de aceptación académica . . . 56

19. Proyecto 17: Sistema inteligente de inspección visual de productos agrícolas 57
19.1. Descripción y objetivo . . . 57
19.2. Usuarios, alcance y requisitos sugeridos . . . 57
19.3. Arquitectura propuesta . . . 57
19.4. Tecnologías sugeridas . . . 57
19.5. Paso a paso de implementación . . . 58
19.6. Diagrama del flujo de extremo a extremo . . . 59
19.7. Plan de pruebas . . . 59
19.8. Entregables específicos . . . 59
19.9. Criterios de aceptación académica . . . 59

20. Proyecto 18: Plataforma de gestión de voluntariado con asignación inteligente 60
20.1. Descripción y objetivo . . . 60
20.2. Usuarios, alcance y requisitos sugeridos . . . 60
20.3. Arquitectura propuesta . . . 60
20.4. Tecnologías sugeridas . . . 60
20.5. Paso a paso de implementación . . . 61
20.6. Diagrama del flujo de extremo a extremo . . . 62
20.7. Plan de pruebas . . . 62
20.8. Entregables específicos . . . 62
20.9. Criterios de aceptación académica . . . 62

21. Proyecto 19: Sistema inteligente de seguimiento de proyectos de construcción 63
21.1. Descripción y objetivo . . . 63
21.2. Usuarios, alcance y requisitos sugeridos . . . 63
21.3. Arquitectura propuesta . . . 63
21.4. Tecnologías sugeridas . . . 63
21.5. Paso a paso de implementación . . . 64
21.6. Diagrama del flujo de extremo a extremo . . . 65
21.7. Plan de pruebas . . . 65
21.8. Entregables específicos . . . 65
21.9. Criterios de aceptación académica . . . 65

22. Proyecto 20: Plataforma de gestión de centros de cómputo con predicción de capacidad 66
22.1. Descripción y objetivo . . . 66
22.2. Usuarios, alcance y requisitos sugeridos . . . 66
22.3. Arquitectura propuesta . . . 66
22.4. Tecnologías sugeridas . . . 66
22.5. Paso a paso de implementación . . . 67
22.6. Diagrama del flujo de extremo a extremo . . . 68
22.7. Plan de pruebas . . . 68
22.8. Entregables específicos . . . 68
22.9. Criterios de aceptación académica . . . 68

23. Proyecto 21: Sistema de clasificación automática de noticias por temática para hemeroteca 69
23.1. Descripción y objetivo . . . 69
23.2. Usuarios, alcance y requisitos sugeridos . . . 69
23.3. Arquitectura propuesta . . . 69
23.4. Tecnologías sugeridas . . . 69
23.5. Paso a paso de implementación . . . 70
23.6. Diagrama del flujo de extremo a extremo . . . 71
23.7. Plan de pruebas . . . 71
23.8. Entregables específicos . . . 71
23.9. Criterios de aceptación académica . . . 71

24. Proyecto 22: Plataforma inteligente de gestión de compras empresariales **72**
24.1. Descripción y objetivo . . . 72
24.2. Usuarios, alcance y requisitos sugeridos . . . 72
24.3. Arquitectura propuesta . . . 72
24.4. Tecnologías sugeridas . . . 72
24.5. Paso a paso de implementación . . . 73
24.6. Diagrama del flujo de extremo a extremo . . . 74
24.7. Plan de pruebas . . . 74
24.8. Entregables específicos . . . 74
24.9. Criterios de aceptación académica . . . 74

25. Proyecto 23: Sistema de detección de defectos en documentos digitalizados **75**
25.1. Descripción y objetivo . . . 75
25.2. Usuarios, alcance y requisitos sugeridos . . . 75
25.3. Arquitectura propuesta . . . 75
25.4. Tecnologías sugeridas . . . 75
25.5. Paso a paso de implementación . . . 76
25.6. Diagrama del flujo de extremo a extremo . . . 77
25.7. Plan de pruebas . . . 77
25.8. Entregables específicos . . . 77
25.9. Criterios de aceptación académica . . . 77

26. Proyecto 24: Plataforma de gestión de servicios de campo con predicción de duración **78**
26.1. Descripción y objetivo . . . 78
26.2. Usuarios, alcance y requisitos sugeridos . . . 78
26.3. Arquitectura propuesta . . . 78
26.4. Tecnologías sugeridas . . . 78
26.5. Paso a paso de implementación . . . 79
26.6. Diagrama del flujo de extremo a extremo . . . 80
26.7. Plan de pruebas . . . 80
26.8. Entregables específicos . . . 80
26.9. Criterios de aceptación académica . . . 80

27. Cronograma sugerido de 16 semanas **81**

28. Rúbrica sugerida **81**

29. Estructura recomendada del repositorio **81**

30. Diagramas que cada equipo debe incluir **82**

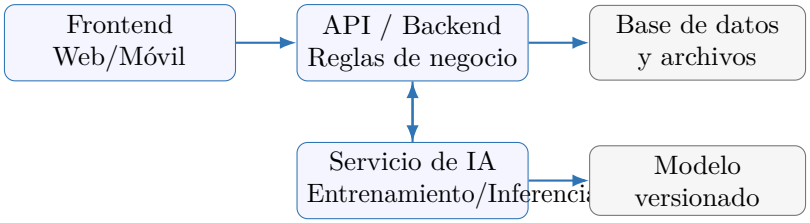
31. Recomendaciones metodológicas **82**

32. Bibliografía base **82**

1. Orientación general del curso

Cada equipo debe entregar un producto de software ejecutable y no solamente un notebook de IA. Se recomienda trabajar en equipos de 3 a 5 estudiantes y organizar el proyecto en iteraciones. La solución debe incluir, como mínimo: levantamiento de requisitos, historias de usuario, criterios de aceptación, arquitectura, modelo de datos, API o capa de servicios, interfaz, módulo de IA, pruebas, control de versiones, documentación técnica y despliegue reproducible.

1.1 Arquitectura de referencia



1.2 Proceso recomendado

1. Definir problema, actores, alcance, restricciones y criterios de éxito.
2. Elaborar backlog con historias de usuario y criterios de aceptación.
3. Diseñar arquitectura, componentes, API, modelo de datos y prototipo UI/UX.
4. Conseguir o construir un dataset lícito, documentado y representativo; separar entrenamiento, validación y prueba.
5. Crear un baseline simple antes de entrenar modelos más complejos.
6. Implementar y evaluar el componente de IA con métricas apropiadas.
7. Integrar el modelo al backend mediante un servicio desacoplado.
8. Implementar frontend, autenticación, roles, validaciones y persistencia.
9. Crear pruebas unitarias, integración, API, sistema y aceptación.
10. Contenerizar, desplegar, monitorizar y documentar reproducibilidad.
11. Analizar riesgos: privacidad, sesgo, seguridad, explicabilidad y uso responsable.
12. Presentar resultados comparando requisitos, métricas de software y métricas de IA.

2. Catálogo resumido

#	Proyecto	IA principal
1	Sistema inteligente de gestión de filas y tiempos de espera	Predicción + optimización de colas
2	Plataforma inteligente de gestión de donaciones y distribución de ayudas	Matching + optimización

#	Proyecto	IA principal
3	Sistema de monitoreo inteligente de redes informáticas	Detección de anomalías + clasificación
4	Plataforma de gestión de restaurantes con predicción de demanda	Forecasting
5	Sistema inteligente para gestión de préstamos de equipos universitarios	Clasificación de riesgo operativo + reglas
6	Plataforma inteligente de análisis de reuniones	Speech/NLP + resumen
7	Sistema de gestión de mantenimiento de software con priorización de deuda técnica	ML + métricas de código
8	Plataforma inteligente para análisis de experiencia del cliente omnicanal	NLP + topic modeling
9	Sistema de monitoreo de cultivos con predicción de necesidades de riego	IoT + regresión/series temporales
10	Plataforma de gestión de gimnasios con recomendación de actividades	Recomendación
11	Sistema inteligente de análisis de documentos de soporte técnico	RAG + búsqueda semántica
12	Plataforma de administración de condominios con clasificación de solicitudes	NLP + workflow
13	Sistema inteligente de control de cadena de frío	IoT + detección de anomalías
14	Plataforma de análisis de tráfico vehicular mediante visión artificial	Computer Vision + series temporales
15	Sistema inteligente de clasificación de correos corporativos	NLP + clasificación
16	Plataforma de gestión de laboratorios con predicción de utilización	Forecasting + optimización
17	Sistema inteligente de inspección visual de productos agrícolas	Computer Vision
18	Plataforma de gestión de voluntariado con asignación inteligente	Matching/recomendación
19	Sistema inteligente de seguimiento de proyectos de construcción	Visión + analítica de avance
20	Plataforma de gestión de centros de cómputo con predicción de capacidad	Forecasting + anomalías
21	Sistema de clasificación automática de noticias por temática para hemeroteca	NLP + clasificación temática
22	Plataforma inteligente de gestión de compras empresariales	Forecasting + recomendación
23	Sistema de detección de defectos en documentos digitalizados	Visión artificial + OCR quality assessment
24	Plataforma de gestión de servicios de campo con predicción de duración	Regresión + planificación

3. Proyecto 1: Sistema inteligente de gestión de filas y tiempos de espera

Enfoque de IA

Predicción + optimización de colas

3.1 Descripción y objetivo

Administrar turnos de atención presencial o virtual, estimar tiempos de espera y recomendar distribución de capacidad según demanda histórica. La propuesta debe desarrollarse como un sistema de software completo. El componente inteligente debe integrarse mediante interfaces bien definidas, registrar su versión y poder evaluarse independientemente de la interfaz de usuario.

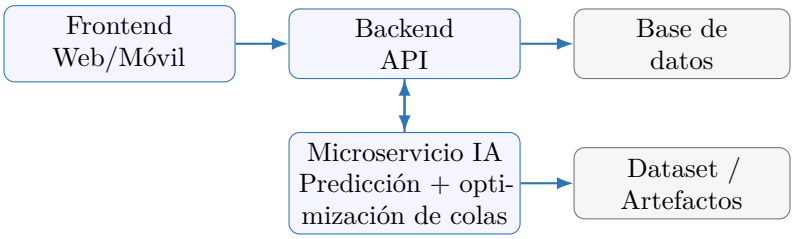
3.2 Usuarios, alcance y requisitos sugeridos

Actores sugeridos: usuario final, operador especializado y administrador. Módulos mínimos:

- Usuarios y sedes
- Generación de turnos
- Ventanillas/servidores
- Predicción de espera
- Alertas
- Dashboard de operación

Como requisitos no funcionales se deben considerar seguridad, validación, disponibilidad, manejo de errores, auditoría, rendimiento, mantenibilidad, accesibilidad, protección de datos, pruebas y documentación.

3.3 Arquitectura propuesta



El backend concentra reglas de negocio y autorización; el servicio de IA recibe entradas validadas y devuelve predicciones con metadatos. Esta separación facilita pruebas, versionado, observabilidad y reemplazo del modelo.

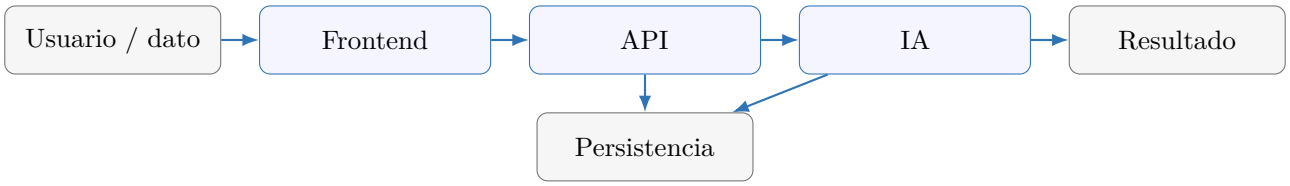
3.4 Tecnologías sugeridas

Python, FastAPI, PostgreSQL, React, scikit-learn, Docker. El equipo puede sustituirlas justificando equivalencia funcional y arquitectónica.

3.5 Paso a paso de implementación

1. **Problema y visión.** Delimitar necesidad, usuarios, objetivos, alcance, exclusiones y métricas de éxito.
2. **Ingeniería de requisitos.** Crear épicas e historias de usuario, criterios de aceptación, requisitos no funcionales y matriz de trazabilidad.
3. **Análisis y dominio.** Identificar entidades, reglas, eventos, estados, excepciones y responsabilidades de los actores.
4. **Prototipado.** Diseñar wireframes del flujo principal y validarlos antes de programar.
5. **Arquitectura.** Crear diagramas de contexto, contenedores/componentes, secuencia y despliegue; especificar endpoints.
6. **Diseño de datos.** Definir modelo lógico, restricciones, índices, migraciones y política de auditoría.
7. **Dataset.** Documentar procedencia, licencia, variables, distribución, calidad, faltantes, sesgos y criterios de partición.
8. **EDA y baseline.** Explorar los datos y construir un modelo base sencillo que establezca una referencia cuantitativa.
9. **Entrenamiento.** Construir pipeline reproducible, seleccionar hiperparámetros usando validación y conservar un test independiente.
10. **Evaluación IA.** Medir desempeño global y por subgrupos pertinentes, analizar errores y documentar limitaciones.
11. **Backend.** Implementar lógica, persistencia, seguridad, validación, manejo de excepciones y documentación OpenAPI.
12. **Servicio IA.** Exponer inferencia mediante interfaz desacoplada y registrar versión, fecha y parámetros necesarios para trazabilidad.
13. **Frontend.** Implementar formularios, dashboards, estados de error/carga y explicación apropiada de resultados.
14. **Integración.** Ejecutar casos de extremo a extremo y verificar consistencia entre UI, API, datos y modelo.
15. **Testing.** Implementar pruebas unitarias, integración, contrato/API, sistema, aceptación y pruebas específicas para datos/modelo.
16. **DevOps/MLOps.** Configurar repositorio, CI, dependencias, variables de entorno, Docker, logging y versionado de modelos.
17. **Resultados.** Reportar MAE del tiempo de espera; tiempo promedio en cola; utilización; abandono; latencia API. Comparar baseline y modelo final e interpretar los errores.
18. **Seguridad y uso responsable.** Crear análisis de amenazas, privacidad, sesgo, abuso, supervisión humana y recuperación ante fallos.
19. **Despliegue.** Preparar ambiente reproducible, datos demo, health checks y procedimiento de instalación.
20. **Cierre.** Elaborar informe, manuales, presentación, demostración y retrospectiva técnica.

3.6 Diagrama del flujo de extremo a extremo



3.7 Plan de pruebas

El equipo debe diseñar pruebas unitarias para servicios y reglas; integración para persistencia y servicio IA; pruebas de API; casos de aceptación derivados de historias de usuario; pruebas de entradas inválidas; evaluación del modelo con test independiente; y una prueba básica de rendimiento. Los defectos encontrados deben registrarse y relacionarse con la corrección aplicada.

3.8 Entregables específicos

Visión y alcance, SRS/backlog, matriz de trazabilidad, prototipos, diagramas TikZ/UML/C4, modelo de datos, repositorio Git, aplicación ejecutable, servicio IA, dataset o instrucciones reproducibles, modelo versionado, pruebas, documentación OpenAPI, Docker, manual técnico, manual de usuario, informe de métricas y video/demo.

3.9 Criterios de aceptación académica

El flujo principal debe funcionar de extremo a extremo y ser demostrable con datos de prueba. La IA debe estar realmente integrada y comparada contra un baseline. Los resultados deben ser reproducibles; los requisitos deben vincularse con pruebas; y el sistema debe manejar errores sin depender de modificaciones manuales del código durante la demostración.

4. Proyecto 2: Plataforma inteligente de gestión de donaciones y distribución de ayudas

Enfoque de IA

Matching + optimización

4.1 Descripción y objetivo

Gestionar donantes, inventario de ayudas, solicitudes y entregas, proponiendo asignaciones según disponibilidad, restricciones y criterios transparentes definidos por la organización. La propuesta debe desarrollarse como un sistema de software completo. El componente inteligente debe integrarse mediante interfaces bien definidas, registrar su versión y poder evaluarse independientemente de la interfaz de usuario.

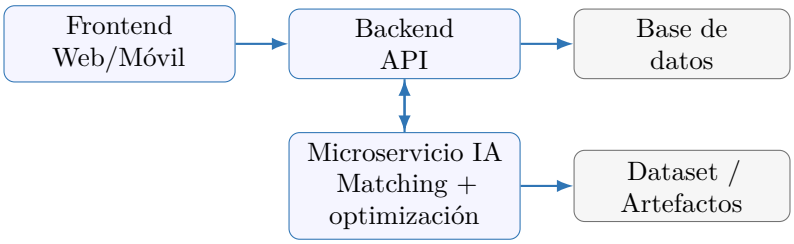
4.2 Usuarios, alcance y requisitos sugeridos

Actores sugeridos: usuario final, operador especializado y administrador. Módulos mínimos:

- Donantes
- Inventario
- Solicitudes
- Motor de matching
- Despachos
- Trazabilidad

Como requisitos no funcionales se deben considerar seguridad, validación, disponibilidad, manejo de errores, auditoría, rendimiento, mantenibilidad, accesibilidad, protección de datos, pruebas y documentación.

4.3 Arquitectura propuesta



El backend concentra reglas de negocio y autorización; el servicio de IA recibe entradas validadas y devuelve predicciones con metadatos. Esta separación facilita pruebas, versionado, observabilidad y reemplazo del modelo.

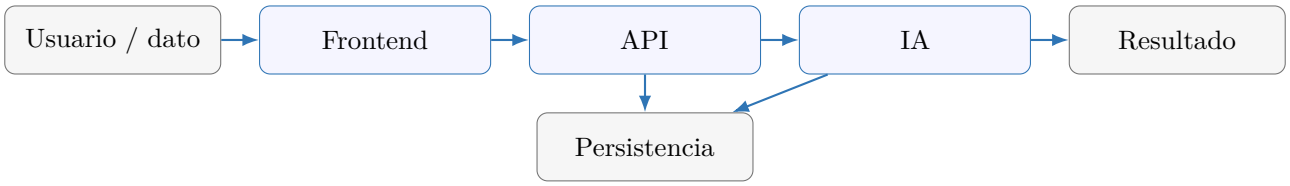
4.4 Tecnologías sugeridas

Python, FastAPI, PostgreSQL, React, OR-Tools, scikit-learn opcional. El equipo puede sustituirlas justificando equivalencia funcional y arquitectónica.

4.5 Paso a paso de implementación

1. **Problema y visión.** Delimitar necesidad, usuarios, objetivos, alcance, exclusiones y métricas de éxito.
2. **Ingeniería de requisitos.** Crear épicas e historias de usuario, criterios de aceptación, requisitos no funcionales y matriz de trazabilidad.
3. **Análisis y dominio.** Identificar entidades, reglas, eventos, estados, excepciones y responsabilidades de los actores.
4. **Prototipado.** Diseñar wireframes del flujo principal y validarlos antes de programar.
5. **Arquitectura.** Crear diagramas de contexto, contenedores/componentes, secuencia y despliegue; especificar endpoints.
6. **Diseño de datos.** Definir modelo lógico, restricciones, índices, migraciones y política de auditoría.
7. **Dataset.** Documentar procedencia, licencia, variables, distribución, calidad, faltantes, sesgos y criterios de partición.
8. **EDA y baseline.** Explorar los datos y construir un modelo base sencillo que establezca una referencia cuantitativa.
9. **Entrenamiento.** Construir pipeline reproducible, seleccionar hiperparámetros usando validación y conservar un test independiente.
10. **Evaluación IA.** Medir desempeño global y por subgrupos pertinentes, analizar errores y documentar limitaciones.
11. **Backend.** Implementar lógica, persistencia, seguridad, validación, manejo de excepciones y documentación OpenAPI.
12. **Servicio IA.** Exponer inferencia mediante interfaz desacoplada y registrar versión, fecha y parámetros necesarios para trazabilidad.
13. **Frontend.** Implementar formularios, dashboards, estados de error/carga y explicación apropiada de resultados.
14. **Integración.** Ejecutar casos de extremo a extremo y verificar consistencia entre UI, API, datos y modelo.
15. **Testing.** Implementar pruebas unitarias, integración, contrato/API, sistema, aceptación y pruebas específicas para datos/modelo.
16. **DevOps/MLOps.** Configurar repositorio, CI, dependencias, variables de entorno, Docker, logging y versionado de modelos.
17. **Resultados.** Reportar Porcentaje de solicitudes cubiertas; tiempo de asignación; utilización de inventario; restricciones incumplidas. Comparar baseline y modelo final e interpretar los errores.
18. **Seguridad y uso responsable.** Crear análisis de amenazas, privacidad, sesgo, abuso, supervisión humana y recuperación ante fallos.
19. **Despliegue.** Preparar ambiente reproducible, datos demo, health checks y procedimiento de instalación.
20. **Cierre.** Elaborar informe, manuales, presentación, demostración y retrospectiva técnica.

4.6 Diagrama del flujo de extremo a extremo



4.7 Plan de pruebas

El equipo debe diseñar pruebas unitarias para servicios y reglas; integración para persistencia y servicio IA; pruebas de API; casos de aceptación derivados de historias de usuario; pruebas de entradas inválidas; evaluación del modelo con test independiente; y una prueba básica de rendimiento. Los defectos encontrados deben registrarse y relacionarse con la corrección aplicada.

4.8 Entregables específicos

Visión y alcance, SRS/backlog, matriz de trazabilidad, prototipos, diagramas TikZ/UML/C4, modelo de datos, repositorio Git, aplicación ejecutable, servicio IA, dataset o instrucciones reproducibles, modelo versionado, pruebas, documentación OpenAPI, Docker, manual técnico, manual de usuario, informe de métricas y video/demo.

4.9 Criterios de aceptación académica

El flujo principal debe funcionar de extremo a extremo y ser demostrable con datos de prueba. La IA debe estar realmente integrada y comparada contra un baseline. Los resultados deben ser reproducibles; los requisitos deben vincularse con pruebas; y el sistema debe manejar errores sin depender de modificaciones manuales del código durante la demostración.

5. Proyecto 3: Sistema de monitoreo inteligente de redes informáticas

Enfoque de IA

Detección de anomalías + clasificación

5.1 Descripción y objetivo

Centralizar métricas de red y detectar patrones anómalos de disponibilidad, latencia o tráfico para apoyar diagnóstico técnico. La propuesta debe desarrollarse como un sistema de software completo. El componente inteligente debe integrarse mediante interfaces bien definidas, registrar su versión y poder evaluarse independientemente de la interfaz de usuario.

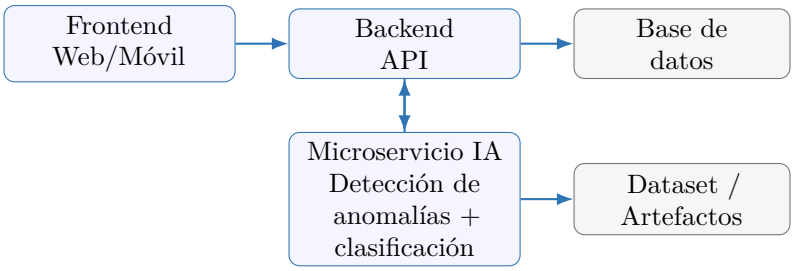
5.2 Usuarios, alcance y requisitos sugeridos

Actores sugeridos: usuario final, operador especializado y administrador. Módulos mínimos:

- Dispositivos
- Telemetría
- Series temporales
- Detección de anomalías
- Incidentes
- Dashboard

Como requisitos no funcionales se deben considerar seguridad, validación, disponibilidad, manejo de errores, auditoría, rendimiento, mantenibilidad, accesibilidad, protección de datos, pruebas y documentación.

5.3 Arquitectura propuesta



El backend concentra reglas de negocio y autorización; el servicio de IA recibe entradas validadas y devuelve predicciones con metadatos. Esta separación facilita pruebas, versionado, observabilidad y reemplazo del modelo.

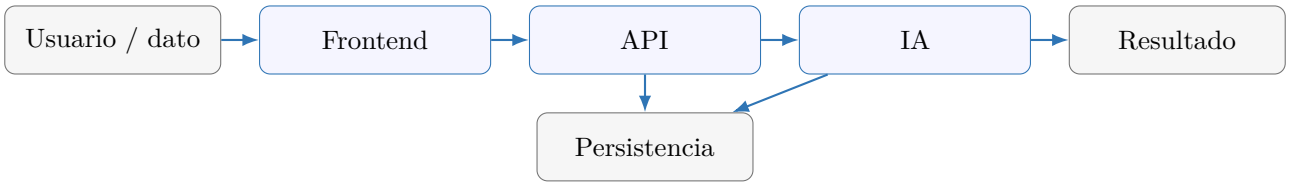
5.4 Tecnologías sugeridas

Python, FastAPI, TimescaleDB, SNMP/telemetría, scikit-learn, Grafana/React. El equipo puede sustituirlas justificando equivalencia funcional y arquitectónica.

5.5 Paso a paso de implementación

1. **Problema y visión.** Delimitar necesidad, usuarios, objetivos, alcance, exclusiones y métricas de éxito.
2. **Ingeniería de requisitos.** Crear épicas e historias de usuario, criterios de aceptación, requisitos no funcionales y matriz de trazabilidad.
3. **Análisis y dominio.** Identificar entidades, reglas, eventos, estados, excepciones y responsabilidades de los actores.
4. **Prototipado.** Diseñar wireframes del flujo principal y validarlos antes de programar.
5. **Arquitectura.** Crear diagramas de contexto, contenedores/componentes, secuencia y despliegue; especificar endpoints.
6. **Diseño de datos.** Definir modelo lógico, restricciones, índices, migraciones y política de auditoría.
7. **Dataset.** Documentar procedencia, licencia, variables, distribución, calidad, faltantes, sesgos y criterios de partición.
8. **EDA y baseline.** Explorar los datos y construir un modelo base sencillo que establezca una referencia cuantitativa.
9. **Entrenamiento.** Construir pipeline reproducible, seleccionar hiperparámetros usando validación y conservar un test independiente.
10. **Evaluación IA.** Medir desempeño global y por subgrupos pertinentes, analizar errores y documentar limitaciones.
11. **Backend.** Implementar lógica, persistencia, seguridad, validación, manejo de excepciones y documentación OpenAPI.
12. **Servicio IA.** Exponer inferencia mediante interfaz desacoplada y registrar versión, fecha y parámetros necesarios para trazabilidad.
13. **Frontend.** Implementar formularios, dashboards, estados de error/carga y explicación apropiada de resultados.
14. **Integración.** Ejecutar casos de extremo a extremo y verificar consistencia entre UI, API, datos y modelo.
15. **Testing.** Implementar pruebas unitarias, integración, contrato/API, sistema, aceptación y pruebas específicas para datos/modelo.
16. **DevOps/MLOps.** Configurar repositorio, CI, dependencias, variables de entorno, Docker, logging y versionado de modelos.
17. **Resultados.** Reportar Precision/recall de anomalías; MTTD; falsas alarmas; disponibilidad; latencia de ingesta. Comparar baseline y modelo final e interpretar los errores.
18. **Seguridad y uso responsable.** Crear análisis de amenazas, privacidad, sesgo, abuso, supervisión humana y recuperación ante fallos.
19. **Despliegue.** Preparar ambiente reproducible, datos demo, health checks y procedimiento de instalación.
20. **Cierre.** Elaborar informe, manuales, presentación, demostración y retrospectiva técnica.

5.6 Diagrama del flujo de extremo a extremo



5.7 Plan de pruebas

El equipo debe diseñar pruebas unitarias para servicios y reglas; integración para persistencia y servicio IA; pruebas de API; casos de aceptación derivados de historias de usuario; pruebas de entradas inválidas; evaluación del modelo con test independiente; y una prueba básica de rendimiento. Los defectos encontrados deben registrarse y relacionarse con la corrección aplicada.

5.8 Entregables específicos

Visión y alcance, SRS/backlog, matriz de trazabilidad, prototipos, diagramas TikZ/UML/C4, modelo de datos, repositorio Git, aplicación ejecutable, servicio IA, dataset o instrucciones reproducibles, modelo versionado, pruebas, documentación OpenAPI, Docker, manual técnico, manual de usuario, informe de métricas y video/demo.

5.9 Criterios de aceptación académica

El flujo principal debe funcionar de extremo a extremo y ser demostrable con datos de prueba. La IA debe estar realmente integrada y comparada contra un baseline. Los resultados deben ser reproducibles; los requisitos deben vincularse con pruebas; y el sistema debe manejar errores sin depender de modificaciones manuales del código durante la demostración.

6. Proyecto 4: Plataforma de gestión de restaurantes con predicción de demanda

Enfoque de IA

Forecasting

6.1 Descripción y objetivo

Gestionar menú, pedidos, mesas e insumos y pronosticar demanda por producto y franja para apoyar preparación y compras. La propuesta debe desarrollarse como un sistema de software completo. El componente inteligente debe integrarse mediante interfaces bien definidas, registrar su versión y poder evaluarse independientemente de la interfaz de usuario.

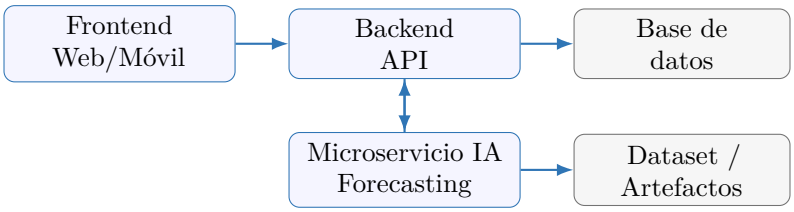
6.2 Usuarios, alcance y requisitos sugeridos

Actores sugeridos: usuario final, operador especializado y administrador. Módulos mínimos:

- Menú
- Pedidos
- Mesas
- Inventario
- Forecast
- Compras sugeridas

Como requisitos no funcionales se deben considerar seguridad, validación, disponibilidad, manejo de errores, auditoría, rendimiento, mantenibilidad, accesibilidad, protección de datos, pruebas y documentación.

6.3 Arquitectura propuesta



El backend concentra reglas de negocio y autorización; el servicio de IA recibe entradas validadas y devuelve predicciones con metadatos. Esta separación facilita pruebas, versionado, observabilidad y reemplazo del modelo.

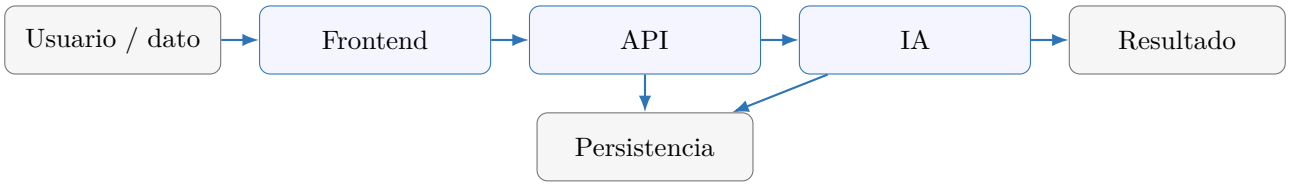
6.4 Tecnologías sugeridas

Python, FastAPI, PostgreSQL, React, statsmodels/scikit-learn. El equipo puede sustituirlas justificando equivalencia funcional y arquitectónica.

6.5 Paso a paso de implementación

1. **Problema y visión.** Delimitar necesidad, usuarios, objetivos, alcance, exclusiones y métricas de éxito.
2. **Ingeniería de requisitos.** Crear épicas e historias de usuario, criterios de aceptación, requisitos no funcionales y matriz de trazabilidad.
3. **Análisis y dominio.** Identificar entidades, reglas, eventos, estados, excepciones y responsabilidades de los actores.
4. **Prototipado.** Diseñar wireframes del flujo principal y validarlos antes de programar.
5. **Arquitectura.** Crear diagramas de contexto, contenedores/componentes, secuencia y despliegue; especificar endpoints.
6. **Diseño de datos.** Definir modelo lógico, restricciones, índices, migraciones y política de auditoría.
7. **Dataset.** Documentar procedencia, licencia, variables, distribución, calidad, faltantes, sesgos y criterios de partición.
8. **EDA y baseline.** Explorar los datos y construir un modelo base sencillo que establezca una referencia cuantitativa.
9. **Entrenamiento.** Construir pipeline reproducible, seleccionar hiperparámetros usando validación y conservar un test independiente.
10. **Evaluación IA.** Medir desempeño global y por subgrupos pertinentes, analizar errores y documentar limitaciones.
11. **Backend.** Implementar lógica, persistencia, seguridad, validación, manejo de excepciones y documentación OpenAPI.
12. **Servicio IA.** Exponer inferencia mediante interfaz desacoplada y registrar versión, fecha y parámetros necesarios para trazabilidad.
13. **Frontend.** Implementar formularios, dashboards, estados de error/carga y explicación apropiada de resultados.
14. **Integración.** Ejecutar casos de extremo a extremo y verificar consistencia entre UI, API, datos y modelo.
15. **Testing.** Implementar pruebas unitarias, integración, contrato/API, sistema, aceptación y pruebas específicas para datos/modelo.
16. **DevOps/MLOps.** Configurar repositorio, CI, dependencias, variables de entorno, Docker, logging y versionado de modelos.
17. **Resultados.** Reportar MAE/sMAPE por producto; desperdicio; faltantes; tiempo de atención; error por horizonte. Comparar baseline y modelo final e interpretar los errores.
18. **Seguridad y uso responsable.** Crear análisis de amenazas, privacidad, sesgo, abuso, supervisión humana y recuperación ante fallos.
19. **Despliegue.** Preparar ambiente reproducible, datos demo, health checks y procedimiento de instalación.
20. **Cierre.** Elaborar informe, manuales, presentación, demostración y retrospectiva técnica.

6.6 Diagrama del flujo de extremo a extremo



6.7 Plan de pruebas

El equipo debe diseñar pruebas unitarias para servicios y reglas; integración para persistencia y servicio IA; pruebas de API; casos de aceptación derivados de historias de usuario; pruebas de entradas inválidas; evaluación del modelo con test independiente; y una prueba básica de rendimiento. Los defectos encontrados deben registrarse y relacionarse con la corrección aplicada.

6.8 Entregables específicos

Visión y alcance, SRS/backlog, matriz de trazabilidad, prototipos, diagramas TikZ/UML/C4, modelo de datos, repositorio Git, aplicación ejecutable, servicio IA, dataset o instrucciones reproducibles, modelo versionado, pruebas, documentación OpenAPI, Docker, manual técnico, manual de usuario, informe de métricas y video/demo.

6.9 Criterios de aceptación académica

El flujo principal debe funcionar de extremo a extremo y ser demostrable con datos de prueba. La IA debe estar realmente integrada y comparada contra un baseline. Los resultados deben ser reproducibles; los requisitos deben vincularse con pruebas; y el sistema debe manejar errores sin depender de modificaciones manuales del código durante la demostración.

7. Proyecto 5: Sistema inteligente para gestión de préstamos de equipos universitarios

Enfoque de IA

Clasificación de riesgo operativo + reglas

7.1 Descripción y objetivo

Administrar inventario, reservas, préstamos y devoluciones, estimando probabilidad de devolución tardía exclusivamente con variables operativas pertinentes. La propuesta debe desarrollarse como un sistema de software completo. El componente inteligente debe integrarse mediante interfaces bien definidas, registrar su versión y poder evaluarse independientemente de la interfaz de usuario.

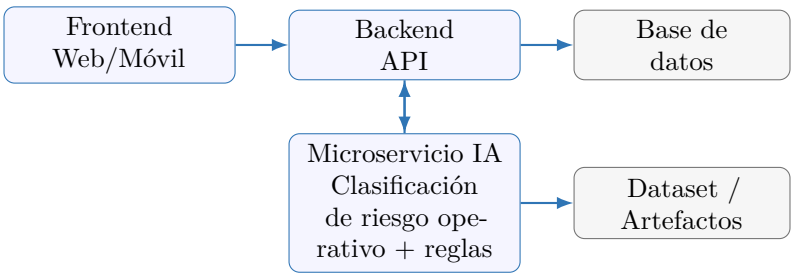
7.2 Usuarios, alcance y requisitos sugeridos

Actores sugeridos: usuario final, operador especializado y administrador. Módulos mínimos:

- Equipos
- Usuarios
- Reservas
- Préstamos
- Alertas
- Mantenimiento

Como requisitos no funcionales se deben considerar seguridad, validación, disponibilidad, manejo de errores, auditoría, rendimiento, mantenibilidad, accesibilidad, protección de datos, pruebas y documentación.

7.3 Arquitectura propuesta



El backend concentra reglas de negocio y autorización; el servicio de IA recibe entradas validadas y devuelve predicciones con metadatos. Esta separación facilita pruebas, versionado, observabilidad y reemplazo del modelo.

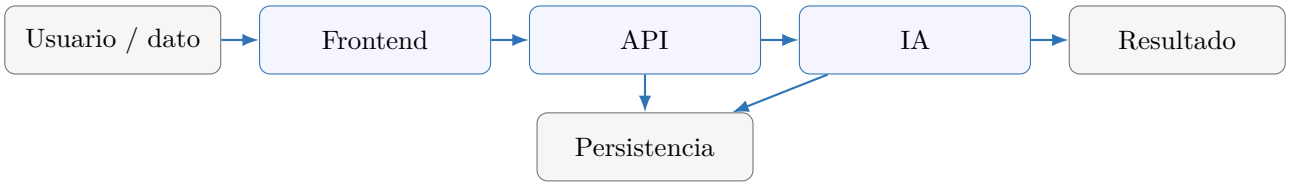
7.4 Tecnologías sugeridas

Python, Django/FastAPI, PostgreSQL, React, scikit-learn. El equipo puede sustituirlas justificando equivalencia funcional y arquitectónica.

7.5 Paso a paso de implementación

1. **Problema y visión.** Delimitar necesidad, usuarios, objetivos, alcance, exclusiones y métricas de éxito.
2. **Ingeniería de requisitos.** Crear épicas e historias de usuario, criterios de aceptación, requisitos no funcionales y matriz de trazabilidad.
3. **Análisis y dominio.** Identificar entidades, reglas, eventos, estados, excepciones y responsabilidades de los actores.
4. **Prototipado.** Diseñar wireframes del flujo principal y validarlos antes de programar.
5. **Arquitectura.** Crear diagramas de contexto, contenedores/componentes, secuencia y despliegue; especificar endpoints.
6. **Diseño de datos.** Definir modelo lógico, restricciones, índices, migraciones y política de auditoría.
7. **Dataset.** Documentar procedencia, licencia, variables, distribución, calidad, faltantes, sesgos y criterios de partición.
8. **EDA y baseline.** Explorar los datos y construir un modelo base sencillo que establezca una referencia cuantitativa.
9. **Entrenamiento.** Construir pipeline reproducible, seleccionar hiperparámetros usando validación y conservar un test independiente.
10. **Evaluación IA.** Medir desempeño global y por subgrupos pertinentes, analizar errores y documentar limitaciones.
11. **Backend.** Implementar lógica, persistencia, seguridad, validación, manejo de excepciones y documentación OpenAPI.
12. **Servicio IA.** Exponer inferencia mediante interfaz desacoplada y registrar versión, fecha y parámetros necesarios para trazabilidad.
13. **Frontend.** Implementar formularios, dashboards, estados de error/carga y explicación apropiada de resultados.
14. **Integración.** Ejecutar casos de extremo a extremo y verificar consistencia entre UI, API, datos y modelo.
15. **Testing.** Implementar pruebas unitarias, integración, contrato/API, sistema, aceptación y pruebas específicas para datos/modelo.
16. **DevOps/MLOps.** Configurar repositorio, CI, dependencias, variables de entorno, Docker, logging y versionado de modelos.
17. **Resultados.** Reportar PR-AUC/recall de retrasos; tasa de devoluciones tardías; disponibilidad; utilización de equipos. Comparar baseline y modelo final e interpretar los errores.
18. **Seguridad y uso responsable.** Crear análisis de amenazas, privacidad, sesgo, abuso, supervisión humana y recuperación ante fallos.
19. **Despliegue.** Preparar ambiente reproducible, datos demo, health checks y procedimiento de instalación.
20. **Cierre.** Elaborar informe, manuales, presentación, demostración y retrospectiva técnica.

7.6 Diagrama del flujo de extremo a extremo



7.7 Plan de pruebas

El equipo debe diseñar pruebas unitarias para servicios y reglas; integración para persistencia y servicio IA; pruebas de API; casos de aceptación derivados de historias de usuario; pruebas de entradas inválidas; evaluación del modelo con test independiente; y una prueba básica de rendimiento. Los defectos encontrados deben registrarse y relacionarse con la corrección aplicada.

7.8 Entregables específicos

Visión y alcance, SRS/backlog, matriz de trazabilidad, prototipos, diagramas TikZ/UML/C4, modelo de datos, repositorio Git, aplicación ejecutable, servicio IA, dataset o instrucciones reproducibles, modelo versionado, pruebas, documentación OpenAPI, Docker, manual técnico, manual de usuario, informe de métricas y video/demo.

7.9 Criterios de aceptación académica

El flujo principal debe funcionar de extremo a extremo y ser demostrable con datos de prueba. La IA debe estar realmente integrada y comparada contra un baseline. Los resultados deben ser reproducibles; los requisitos deben vincularse con pruebas; y el sistema debe manejar errores sin depender de modificaciones manuales del código durante la demostración.

8. Proyecto 6: Plataforma inteligente de análisis de reuniones

Enfoque de IA

Speech/NLP + resumen

8.1 Descripción y objetivo

Gestionar reuniones y, a partir de transcripciones autorizadas, identificar temas, acuerdos, responsables y tareas para mantener trazabilidad. La propuesta debe desarrollarse como un sistema de software completo. El componente inteligente debe integrarse mediante interfaces bien definidas, registrar su versión y poder evaluarse independientemente de la interfaz de usuario.

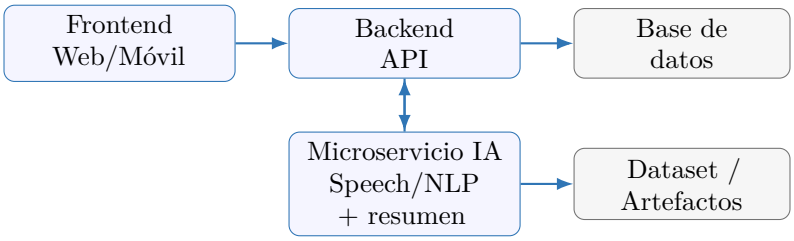
8.2 Usuarios, alcance y requisitos sugeridos

Actores sugeridos: usuario final, operador especializado y administrador. Módulos mínimos:

- Reuniones
- Carga/transcripción
- Segmentación
- Resumen
- Extracción de acuerdos
- Gestión de tareas

Como requisitos no funcionales se deben considerar seguridad, validación, disponibilidad, manejo de errores, auditoría, rendimiento, mantenibilidad, accesibilidad, protección de datos, pruebas y documentación.

8.3 Arquitectura propuesta



El backend concentra reglas de negocio y autorización; el servicio de IA recibe entradas validadas y devuelve predicciones con metadatos. Esta separación facilita pruebas, versionado, observabilidad y reemplazo del modelo.

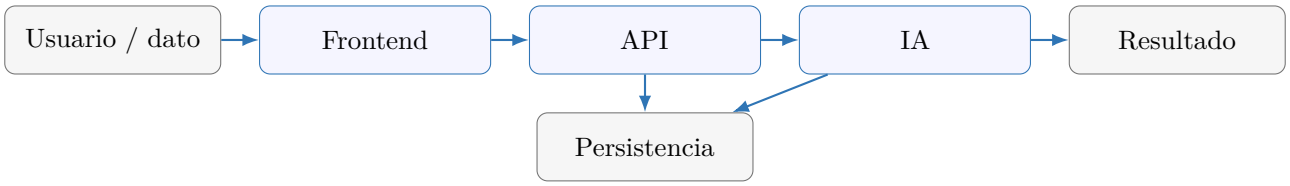
8.4 Tecnologías sugeridas

Python, FastAPI, PostgreSQL, React, modelo ASR configurable, Transformers/LLM. El equipo puede sustituirlas justificando equivalencia funcional y arquitectónica.

8.5 Paso a paso de implementación

1. **Problema y visión.** Delimitar necesidad, usuarios, objetivos, alcance, exclusiones y métricas de éxito.
2. **Ingeniería de requisitos.** Crear épicas e historias de usuario, criterios de aceptación, requisitos no funcionales y matriz de trazabilidad.
3. **Análisis y dominio.** Identificar entidades, reglas, eventos, estados, excepciones y responsabilidades de los actores.
4. **Prototipado.** Diseñar wireframes del flujo principal y validarlos antes de programar.
5. **Arquitectura.** Crear diagramas de contexto, contenedores/componentes, secuencia y despliegue; especificar endpoints.
6. **Diseño de datos.** Definir modelo lógico, restricciones, índices, migraciones y política de auditoría.
7. **Dataset.** Documentar procedencia, licencia, variables, distribución, calidad, faltantes, sesgos y criterios de partición.
8. **EDA y baseline.** Explorar los datos y construir un modelo base sencillo que establezca una referencia cuantitativa.
9. **Entrenamiento.** Construir pipeline reproducible, seleccionar hiperparámetros usando validación y conservar un test independiente.
10. **Evaluación IA.** Medir desempeño global y por subgrupos pertinentes, analizar errores y documentar limitaciones.
11. **Backend.** Implementar lógica, persistencia, seguridad, validación, manejo de excepciones y documentación OpenAPI.
12. **Servicio IA.** Exponer inferencia mediante interfaz desacoplada y registrar versión, fecha y parámetros necesarios para trazabilidad.
13. **Frontend.** Implementar formularios, dashboards, estados de error/carga y explicación apropiada de resultados.
14. **Integración.** Ejecutar casos de extremo a extremo y verificar consistencia entre UI, API, datos y modelo.
15. **Testing.** Implementar pruebas unitarias, integración, contrato/API, sistema, aceptación y pruebas específicas para datos/modelo.
16. **DevOps/MLOps.** Configurar repositorio, CI, dependencias, variables de entorno, Docker, logging y versionado de modelos.
17. **Resultados.** Reportar WER si se evalúa ASR; F1 de extracción de acuerdos; cobertura de tareas; correcciones humanas. Comparar baseline y modelo final e interpretar los errores.
18. **Seguridad y uso responsable.** Crear análisis de amenazas, privacidad, sesgo, abuso, supervisión humana y recuperación ante fallos.
19. **Despliegue.** Preparar ambiente reproducible, datos demo, health checks y procedimiento de instalación.
20. **Cierre.** Elaborar informe, manuales, presentación, demostración y retrospectiva técnica.

8.6 Diagrama del flujo de extremo a extremo



8.7 Plan de pruebas

El equipo debe diseñar pruebas unitarias para servicios y reglas; integración para persistencia y servicio IA; pruebas de API; casos de aceptación derivados de historias de usuario; pruebas de entradas inválidas; evaluación del modelo con test independiente; y una prueba básica de rendimiento. Los defectos encontrados deben registrarse y relacionarse con la corrección aplicada.

8.8 Entregables específicos

Visión y alcance, SRS/backlog, matriz de trazabilidad, prototipos, diagramas TikZ/UML/C4, modelo de datos, repositorio Git, aplicación ejecutable, servicio IA, dataset o instrucciones reproducibles, modelo versionado, pruebas, documentación OpenAPI, Docker, manual técnico, manual de usuario, informe de métricas y video/demo.

8.9 Criterios de aceptación académica

El flujo principal debe funcionar de extremo a extremo y ser demostrable con datos de prueba. La IA debe estar realmente integrada y comparada contra un baseline. Los resultados deben ser reproducibles; los requisitos deben vincularse con pruebas; y el sistema debe manejar errores sin depender de modificaciones manuales del código durante la demostración.

9. Proyecto 7: Sistema de gestión de mantenimiento de software con priorización de deuda técnica

Enfoque de IA

ML + métricas de código

9.1 Descripción y objetivo

Registrar componentes, incidencias y deuda técnica y priorizar elementos usando métricas de mantenibilidad, historial de cambios y defectos. La propuesta debe desarrollarse como un sistema de software completo. El componente inteligente debe integrarse mediante interfaces bien definidas, registrar su versión y poder evaluarse independientemente de la interfaz de usuario.

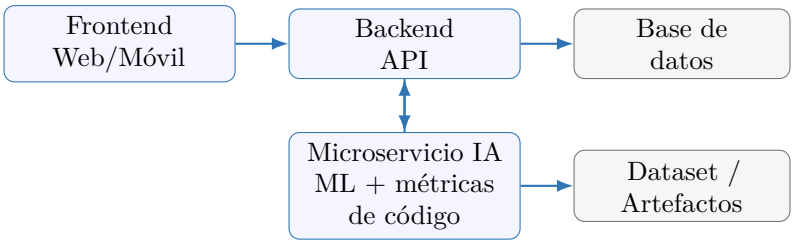
9.2 Usuarios, alcance y requisitos sugeridos

Actores sugeridos: usuario final, operador especializado y administrador. Módulos mínimos:

- Repositorios
- Métricas
- Deuda técnica
- Incidencias
- Priorización
- Dashboard

Como requisitos no funcionales se deben considerar seguridad, validación, disponibilidad, manejo de errores, auditoría, rendimiento, mantenibilidad, accesibilidad, protección de datos, pruebas y documentación.

9.3 Arquitectura propuesta



El backend concentra reglas de negocio y autorización; el servicio de IA recibe entradas validadas y devuelve predicciones con metadatos. Esta separación facilita pruebas, versionado, observabilidad y reemplazo del modelo.

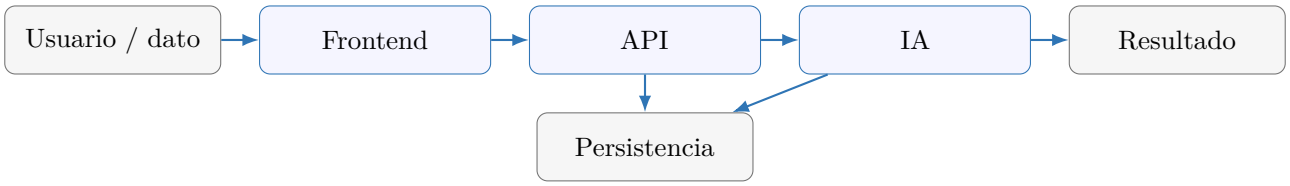
9.4 Tecnologías sugeridas

Python, FastAPI, PostgreSQL, Git APIs, SonarQube, scikit-learn. El equipo puede sustituirlas justificando equivalencia funcional y arquitectónica.

9.5 Paso a paso de implementación

1. **Problema y visión.** Delimitar necesidad, usuarios, objetivos, alcance, exclusiones y métricas de éxito.
2. **Ingeniería de requisitos.** Crear épicas e historias de usuario, criterios de aceptación, requisitos no funcionales y matriz de trazabilidad.
3. **Análisis y dominio.** Identificar entidades, reglas, eventos, estados, excepciones y responsabilidades de los actores.
4. **Prototipado.** Diseñar wireframes del flujo principal y validarlos antes de programar.
5. **Arquitectura.** Crear diagramas de contexto, contenedores/componentes, secuencia y despliegue; especificar endpoints.
6. **Diseño de datos.** Definir modelo lógico, restricciones, índices, migraciones y política de auditoría.
7. **Dataset.** Documentar procedencia, licencia, variables, distribución, calidad, faltantes, sesgos y criterios de partición.
8. **EDA y baseline.** Explorar los datos y construir un modelo base sencillo que establezca una referencia cuantitativa.
9. **Entrenamiento.** Construir pipeline reproducible, seleccionar hiperparámetros usando validación y conservar un test independiente.
10. **Evaluación IA.** Medir desempeño global y por subgrupos pertinentes, analizar errores y documentar limitaciones.
11. **Backend.** Implementar lógica, persistencia, seguridad, validación, manejo de excepciones y documentación OpenAPI.
12. **Servicio IA.** Exponer inferencia mediante interfaz desacoplada y registrar versión, fecha y parámetros necesarios para trazabilidad.
13. **Frontend.** Implementar formularios, dashboards, estados de error/carga y explicación apropiada de resultados.
14. **Integración.** Ejecutar casos de extremo a extremo y verificar consistencia entre UI, API, datos y modelo.
15. **Testing.** Implementar pruebas unitarias, integración, contrato/API, sistema, aceptación y pruebas específicas para datos/modelo.
16. **DevOps/MLOps.** Configurar repositorio, CI, dependencias, variables de entorno, Docker, logging y versionado de modelos.
17. **Resultados.** Reportar Correlación con defectos; precision@K de ítems priorizados; complejidad; tiempo de resolución. Comparar baseline y modelo final e interpretar los errores.
18. **Seguridad y uso responsable.** Crear análisis de amenazas, privacidad, sesgo, abuso, supervisión humana y recuperación ante fallos.
19. **Despliegue.** Preparar ambiente reproducible, datos demo, health checks y procedimiento de instalación.
20. **Cierre.** Elaborar informe, manuales, presentación, demostración y retrospectiva técnica.

9.6 Diagrama del flujo de extremo a extremo



9.7 Plan de pruebas

El equipo debe diseñar pruebas unitarias para servicios y reglas; integración para persistencia y servicio IA; pruebas de API; casos de aceptación derivados de historias de usuario; pruebas de entradas inválidas; evaluación del modelo con test independiente; y una prueba básica de rendimiento. Los defectos encontrados deben registrarse y relacionarse con la corrección aplicada.

9.8 Entregables específicos

Visión y alcance, SRS/backlog, matriz de trazabilidad, prototipos, diagramas TikZ/UML/C4, modelo de datos, repositorio Git, aplicación ejecutable, servicio IA, dataset o instrucciones reproducibles, modelo versionado, pruebas, documentación OpenAPI, Docker, manual técnico, manual de usuario, informe de métricas y video/demo.

9.9 Criterios de aceptación académica

El flujo principal debe funcionar de extremo a extremo y ser demostrable con datos de prueba. La IA debe estar realmente integrada y comparada contra un baseline. Los resultados deben ser reproducibles; los requisitos deben vincularse con pruebas; y el sistema debe manejar errores sin depender de modificaciones manuales del código durante la demostración.

10. Proyecto 8: Plataforma inteligente para análisis de experiencia del cliente omnicanal

Enfoque de IA

NLP + topic modeling

10.1 Descripción y objetivo

Integrar feedback de formularios, chats y encuestas para clasificar motivos, sentimiento y temas, con trazabilidad hacia acciones de mejora. La propuesta debe desarrollarse como un sistema de software completo. El componente inteligente debe integrarse mediante interfaces bien definidas, registrar su versión y poder evaluarse independientemente de la interfaz de usuario.

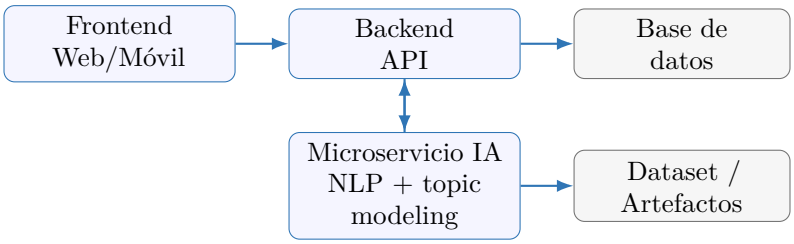
10.2 Usuarios, alcance y requisitos sugeridos

Actores sugeridos: usuario final, operador especializado y administrador. Módulos mínimos:

- Fuentes
- ETL
- Anonimización
- Clasificación
- Tópicos
- Dashboard

Como requisitos no funcionales se deben considerar seguridad, validación, disponibilidad, manejo de errores, auditoría, rendimiento, mantenibilidad, accesibilidad, protección de datos, pruebas y documentación.

10.3 Arquitectura propuesta



El backend concentra reglas de negocio y autorización; el servicio de IA recibe entradas validadas y devuelve predicciones con metadatos. Esta separación facilita pruebas, versionado, observabilidad y reemplazo del modelo.

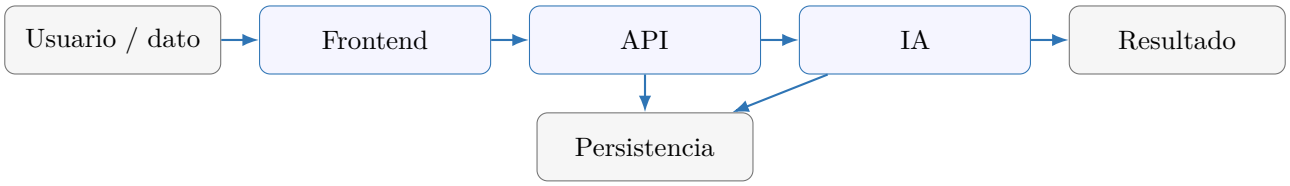
10.4 Tecnologías sugeridas

Python, FastAPI, PostgreSQL, React/Power BI, Transformers, BERTopic opcional. El equipo puede sustituirlas justificando equivalencia funcional y arquitectónica.

10.5 Paso a paso de implementación

1. **Problema y visión.** Delimitar necesidad, usuarios, objetivos, alcance, exclusiones y métricas de éxito.
2. **Ingeniería de requisitos.** Crear épicas e historias de usuario, criterios de aceptación, requisitos no funcionales y matriz de trazabilidad.
3. **Análisis y dominio.** Identificar entidades, reglas, eventos, estados, excepciones y responsabilidades de los actores.
4. **Prototipado.** Diseñar wireframes del flujo principal y validarlos antes de programar.
5. **Arquitectura.** Crear diagramas de contexto, contenedores/componentes, secuencia y despliegue; especificar endpoints.
6. **Diseño de datos.** Definir modelo lógico, restricciones, índices, migraciones y política de auditoría.
7. **Dataset.** Documentar procedencia, licencia, variables, distribución, calidad, faltantes, sesgos y criterios de partición.
8. **EDA y baseline.** Explorar los datos y construir un modelo base sencillo que establezca una referencia cuantitativa.
9. **Entrenamiento.** Construir pipeline reproducible, seleccionar hiperparámetros usando validación y conservar un test independiente.
10. **Evaluación IA.** Medir desempeño global y por subgrupos pertinentes, analizar errores y documentar limitaciones.
11. **Backend.** Implementar lógica, persistencia, seguridad, validación, manejo de excepciones y documentación OpenAPI.
12. **Servicio IA.** Exponer inferencia mediante interfaz desacoplada y registrar versión, fecha y parámetros necesarios para trazabilidad.
13. **Frontend.** Implementar formularios, dashboards, estados de error/carga y explicación apropiada de resultados.
14. **Integración.** Ejecutar casos de extremo a extremo y verificar consistencia entre UI, API, datos y modelo.
15. **Testing.** Implementar pruebas unitarias, integración, contrato/API, sistema, aceptación y pruebas específicas para datos/modelo.
16. **DevOps/MLOps.** Configurar repositorio, CI, dependencias, variables de entorno, Docker, logging y versionado de modelos.
17. **Resultados.** Reportar Macro-F1; coherencia temática; cobertura de feedback; latencia; tasa de casos accionables. Comparar baseline y modelo final e interpretar los errores.
18. **Seguridad y uso responsable.** Crear análisis de amenazas, privacidad, sesgo, abuso, supervisión humana y recuperación ante fallos.
19. **Despliegue.** Preparar ambiente reproducible, datos demo, health checks y procedimiento de instalación.
20. **Cierre.** Elaborar informe, manuales, presentación, demostración y retrospectiva técnica.

10.6 Diagrama del flujo de extremo a extremo



10.7 Plan de pruebas

El equipo debe diseñar pruebas unitarias para servicios y reglas; integración para persistencia y servicio IA; pruebas de API; casos de aceptación derivados de historias de usuario; pruebas de entradas inválidas; evaluación del modelo con test independiente; y una prueba básica de rendimiento. Los defectos encontrados deben registrarse y relacionarse con la corrección aplicada.

10.8 Entregables específicos

Visión y alcance, SRS/backlog, matriz de trazabilidad, prototipos, diagramas TikZ/UML/C4, modelo de datos, repositorio Git, aplicación ejecutable, servicio IA, dataset o instrucciones reproducibles, modelo versionado, pruebas, documentación OpenAPI, Docker, manual técnico, manual de usuario, informe de métricas y video/demo.

10.9 Criterios de aceptación académica

El flujo principal debe funcionar de extremo a extremo y ser demostrable con datos de prueba. La IA debe estar realmente integrada y comparada contra un baseline. Los resultados deben ser reproducibles; los requisitos deben vincularse con pruebas; y el sistema debe manejar errores sin depender de modificaciones manuales del código durante la demostración.

11. Proyecto 9: Sistema de monitoreo de cultivos con predicción de necesidades de riego

Enfoque de IA

IoT + regresión/series temporales

11.1 Descripción y objetivo

Registrar humedad y variables ambientales para estimar necesidades de riego y apoyar decisiones del productor sin controlar automáticamente equipos críticos. La propuesta debe desarrollarse como un sistema de software completo. El componente inteligente debe integrarse mediante interfaces bien definidas, registrar su versión y poder evaluarse independientemente de la interfaz de usuario.

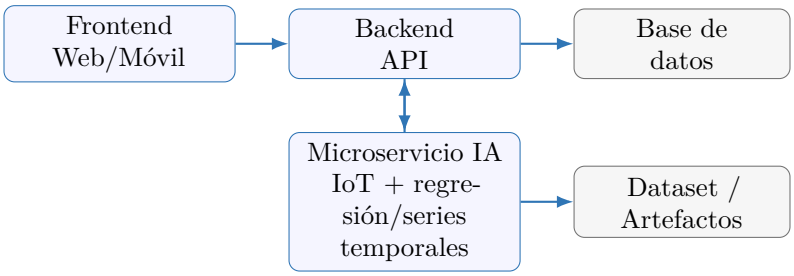
11.2 Usuarios, alcance y requisitos sugeridos

Actores sugeridos: usuario final, operador especializado y administrador. Módulos mínimos:

- Lotes
- Sensores
- Ingesta IoT
- Histórico
- Modelo
- Recomendaciones

Como requisitos no funcionales se deben considerar seguridad, validación, disponibilidad, manejo de errores, auditoría, rendimiento, mantenibilidad, accesibilidad, protección de datos, pruebas y documentación.

11.3 Arquitectura propuesta



El backend concentra reglas de negocio y autorización; el servicio de IA recibe entradas validadas y devuelve predicciones con metadatos. Esta separación facilita pruebas, versionado, observabilidad y reemplazo del modelo.

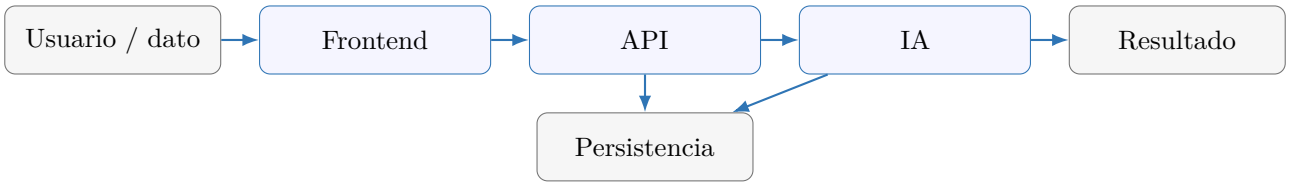
11.4 Tecnologías sugeridas

Python, FastAPI, TimescaleDB, MQTT, scikit-learn, React. El equipo puede sustituirlas justificando equivalencia funcional y arquitectónica.

11.5 Paso a paso de implementación

1. **Problema y visión.** Delimitar necesidad, usuarios, objetivos, alcance, exclusiones y métricas de éxito.
2. **Ingeniería de requisitos.** Crear épicas e historias de usuario, criterios de aceptación, requisitos no funcionales y matriz de trazabilidad.
3. **Análisis y dominio.** Identificar entidades, reglas, eventos, estados, excepciones y responsabilidades de los actores.
4. **Prototipado.** Diseñar wireframes del flujo principal y validarlos antes de programar.
5. **Arquitectura.** Crear diagramas de contexto, contenedores/componentes, secuencia y despliegue; especificar endpoints.
6. **Diseño de datos.** Definir modelo lógico, restricciones, índices, migraciones y política de auditoría.
7. **Dataset.** Documentar procedencia, licencia, variables, distribución, calidad, faltantes, sesgos y criterios de partición.
8. **EDA y baseline.** Explorar los datos y construir un modelo base sencillo que establezca una referencia cuantitativa.
9. **Entrenamiento.** Construir pipeline reproducible, seleccionar hiperparámetros usando validación y conservar un test independiente.
10. **Evaluación IA.** Medir desempeño global y por subgrupos pertinentes, analizar errores y documentar limitaciones.
11. **Backend.** Implementar lógica, persistencia, seguridad, validación, manejo de excepciones y documentación OpenAPI.
12. **Servicio IA.** Exponer inferencia mediante interfaz desacoplada y registrar versión, fecha y parámetros necesarios para trazabilidad.
13. **Frontend.** Implementar formularios, dashboards, estados de error/carga y explicación apropiada de resultados.
14. **Integración.** Ejecutar casos de extremo a extremo y verificar consistencia entre UI, API, datos y modelo.
15. **Testing.** Implementar pruebas unitarias, integración, contrato/API, sistema, aceptación y pruebas específicas para datos/modelo.
16. **DevOps/MLOps.** Configurar repositorio, CI, dependencias, variables de entorno, Docker, logging y versionado de modelos.
17. **Resultados.** Reportar MAE de humedad/consumo; disponibilidad de sensores; ahorro estimado; falsas alertas. Comparar baseline y modelo final e interpretar los errores.
18. **Seguridad y uso responsable.** Crear análisis de amenazas, privacidad, sesgo, abuso, supervisión humana y recuperación ante fallos.
19. **Despliegue.** Preparar ambiente reproducible, datos demo, health checks y procedimiento de instalación.
20. **Cierre.** Elaborar informe, manuales, presentación, demostración y retrospectiva técnica.

11.6 Diagrama del flujo de extremo a extremo



11.7 Plan de pruebas

El equipo debe diseñar pruebas unitarias para servicios y reglas; integración para persistencia y servicio IA; pruebas de API; casos de aceptación derivados de historias de usuario; pruebas de entradas inválidas; evaluación del modelo con test independiente; y una prueba básica de rendimiento. Los defectos encontrados deben registrarse y relacionarse con la corrección aplicada.

11.8 Entregables específicos

Visión y alcance, SRS/backlog, matriz de trazabilidad, prototipos, diagramas TikZ/UML/C4, modelo de datos, repositorio Git, aplicación ejecutable, servicio IA, dataset o instrucciones reproducibles, modelo versionado, pruebas, documentación OpenAPI, Docker, manual técnico, manual de usuario, informe de métricas y video/demo.

11.9 Criterios de aceptación académica

El flujo principal debe funcionar de extremo a extremo y ser demostrable con datos de prueba. La IA debe estar realmente integrada y comparada contra un baseline. Los resultados deben ser reproducibles; los requisitos deben vincularse con pruebas; y el sistema debe manejar errores sin depender de modificaciones manuales del código durante la demostración.

12. Proyecto 10: Plataforma de gestión de gimnasios con recomendación de actividades

Enfoque de IA

Recomendación

12.1 Descripción y objetivo

Gestionar membresías, clases, reservas y asistencia y recomendar actividades según preferencias declaradas, disponibilidad y uso histórico. La propuesta debe desarrollarse como un sistema de software completo. El componente inteligente debe integrarse mediante interfaces bien definidas, registrar su versión y poder evaluarse independientemente de la interfaz de usuario.

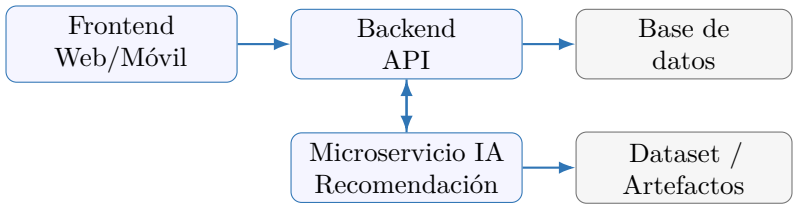
12.2 Usuarios, alcance y requisitos sugeridos

Actores sugeridos: usuario final, operador especializado y administrador. Módulos mínimos:

- Membresías
- Clases
- Reservas
- Asistencia
- Recomendador
- Panel

Como requisitos no funcionales se deben considerar seguridad, validación, disponibilidad, manejo de errores, auditoría, rendimiento, mantenibilidad, accesibilidad, protección de datos, pruebas y documentación.

12.3 Arquitectura propuesta



El backend concentra reglas de negocio y autorización; el servicio de IA recibe entradas validadas y devuelve predicciones con metadatos. Esta separación facilita pruebas, versionado, observabilidad y reemplazo del modelo.

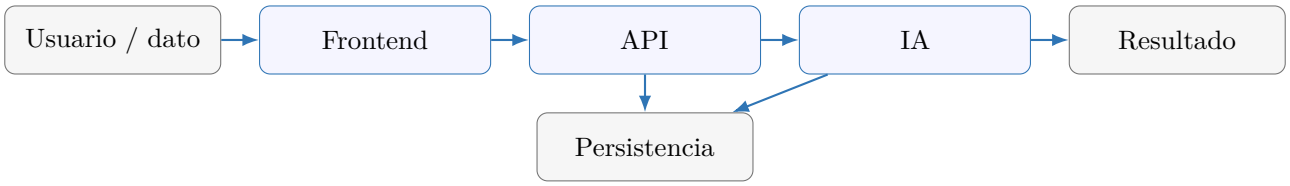
12.4 Tecnologías sugeridas

Python, FastAPI, PostgreSQL, React, scikit-learn. El equipo puede sustituirlas justificando equivalencia funcional y arquitectónica.

12.5 Paso a paso de implementación

1. **Problema y visión.** Delimitar necesidad, usuarios, objetivos, alcance, exclusiones y métricas de éxito.
2. **Ingeniería de requisitos.** Crear épicas e historias de usuario, criterios de aceptación, requisitos no funcionales y matriz de trazabilidad.
3. **Análisis y dominio.** Identificar entidades, reglas, eventos, estados, excepciones y responsabilidades de los actores.
4. **Prototipado.** Diseñar wireframes del flujo principal y validarlos antes de programar.
5. **Arquitectura.** Crear diagramas de contexto, contenedores/componentes, secuencia y despliegue; especificar endpoints.
6. **Diseño de datos.** Definir modelo lógico, restricciones, índices, migraciones y política de auditoría.
7. **Dataset.** Documentar procedencia, licencia, variables, distribución, calidad, faltantes, sesgos y criterios de partición.
8. **EDA y baseline.** Explorar los datos y construir un modelo base sencillo que establezca una referencia cuantitativa.
9. **Entrenamiento.** Construir pipeline reproducible, seleccionar hiperparámetros usando validación y conservar un test independiente.
10. **Evaluación IA.** Medir desempeño global y por subgrupos pertinentes, analizar errores y documentar limitaciones.
11. **Backend.** Implementar lógica, persistencia, seguridad, validación, manejo de excepciones y documentación OpenAPI.
12. **Servicio IA.** Exponer inferencia mediante interfaz desacoplada y registrar versión, fecha y parámetros necesarios para trazabilidad.
13. **Frontend.** Implementar formularios, dashboards, estados de error/carga y explicación apropiada de resultados.
14. **Integración.** Ejecutar casos de extremo a extremo y verificar consistencia entre UI, API, datos y modelo.
15. **Testing.** Implementar pruebas unitarias, integración, contrato/API, sistema, aceptación y pruebas específicas para datos/modelo.
16. **DevOps/MLOps.** Configurar repositorio, CI, dependencias, variables de entorno, Docker, logging y versionado de modelos.
17. **Resultados.** Reportar Precision@K; tasa de reserva desde recomendación; ocupación; diversidad de recomendaciones. Comparar baseline y modelo final e interpretar los errores.
18. **Seguridad y uso responsable.** Crear análisis de amenazas, privacidad, sesgo, abuso, supervisión humana y recuperación ante fallos.
19. **Despliegue.** Preparar ambiente reproducible, datos demo, health checks y procedimiento de instalación.
20. **Cierre.** Elaborar informe, manuales, presentación, demostración y retrospectiva técnica.

12.6 Diagrama del flujo de extremo a extremo



12.7 Plan de pruebas

El equipo debe diseñar pruebas unitarias para servicios y reglas; integración para persistencia y servicio IA; pruebas de API; casos de aceptación derivados de historias de usuario; pruebas de entradas inválidas; evaluación del modelo con test independiente; y una prueba básica de rendimiento. Los defectos encontrados deben registrarse y relacionarse con la corrección aplicada.

12.8 Entregables específicos

Visión y alcance, SRS/backlog, matriz de trazabilidad, prototipos, diagramas TikZ/UML/C4, modelo de datos, repositorio Git, aplicación ejecutable, servicio IA, dataset o instrucciones reproducibles, modelo versionado, pruebas, documentación OpenAPI, Docker, manual técnico, manual de usuario, informe de métricas y video/demo.

12.9 Criterios de aceptación académica

El flujo principal debe funcionar de extremo a extremo y ser demostrable con datos de prueba. La IA debe estar realmente integrada y comparada contra un baseline. Los resultados deben ser reproducibles; los requisitos deben vincularse con pruebas; y el sistema debe manejar errores sin depender de modificaciones manuales del código durante la demostración.

13. Proyecto 11: Sistema inteligente de análisis de documentos de soporte técnico

Enfoque de IA

RAG + búsqueda semántica

13.1 Descripción y objetivo

Administrar manuales, procedimientos y preguntas frecuentes y responder consultas técnicas recuperando fragmentos autorizados y citando sus fuentes. La propuesta debe desarrollarse como un sistema de software completo. El componente inteligente debe integrarse mediante interfaces bien definidas, registrar su versión y poder evaluarse independientemente de la interfaz de usuario.

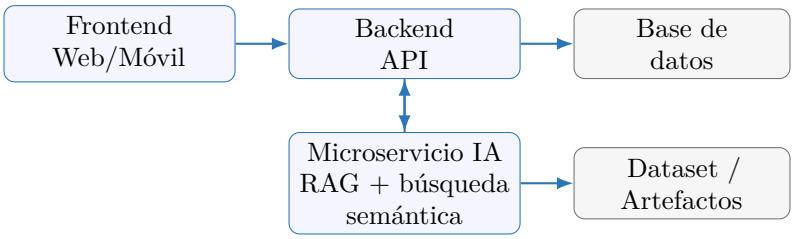
13.2 Usuarios, alcance y requisitos sugeridos

Actores sugeridos: usuario final, operador especializado y administrador. Módulos mínimos:

- Documentos
- Versionado
- Indexación
- Búsqueda semántica
- Respuesta con citas
- Feedback

Como requisitos no funcionales se deben considerar seguridad, validación, disponibilidad, manejo de errores, auditoría, rendimiento, mantenibilidad, accesibilidad, protección de datos, pruebas y documentación.

13.3 Arquitectura propuesta



El backend concentra reglas de negocio y autorización; el servicio de IA recibe entradas validadas y devuelve predicciones con metadatos. Esta separación facilita pruebas, versionado, observabilidad y reemplazo del modelo.

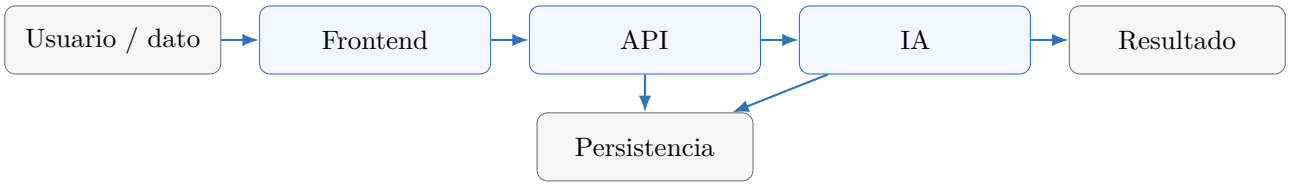
13.4 Tecnologías sugeridas

Python, FastAPI, PostgreSQL, Vector DB, React, Embeddings + LLM. El equipo puede sustituirlas justificando equivalencia funcional y arquitectónica.

13.5 Paso a paso de implementación

1. **Problema y visión.** Delimitar necesidad, usuarios, objetivos, alcance, exclusiones y métricas de éxito.
2. **Ingeniería de requisitos.** Crear épicas e historias de usuario, criterios de aceptación, requisitos no funcionales y matriz de trazabilidad.
3. **Análisis y dominio.** Identificar entidades, reglas, eventos, estados, excepciones y responsabilidades de los actores.
4. **Prototipado.** Diseñar wireframes del flujo principal y validarlos antes de programar.
5. **Arquitectura.** Crear diagramas de contexto, contenedores/componentes, secuencia y despliegue; especificar endpoints.
6. **Diseño de datos.** Definir modelo lógico, restricciones, índices, migraciones y política de auditoría.
7. **Dataset.** Documentar procedencia, licencia, variables, distribución, calidad, faltantes, sesgos y criterios de partición.
8. **EDA y baseline.** Explorar los datos y construir un modelo base sencillo que establezca una referencia cuantitativa.
9. **Entrenamiento.** Construir pipeline reproducible, seleccionar hiperparámetros usando validación y conservar un test independiente.
10. **Evaluación IA.** Medir desempeño global y por subgrupos pertinentes, analizar errores y documentar limitaciones.
11. **Backend.** Implementar lógica, persistencia, seguridad, validación, manejo de excepciones y documentación OpenAPI.
12. **Servicio IA.** Exponer inferencia mediante interfaz desacoplada y registrar versión, fecha y parámetros necesarios para trazabilidad.
13. **Frontend.** Implementar formularios, dashboards, estados de error/carga y explicación apropiada de resultados.
14. **Integración.** Ejecutar casos de extremo a extremo y verificar consistencia entre UI, API, datos y modelo.
15. **Testing.** Implementar pruebas unitarias, integración, contrato/API, sistema, aceptación y pruebas específicas para datos/modelo.
16. **DevOps/MLOps.** Configurar repositorio, CI, dependencias, variables de entorno, Docker, logging y versionado de modelos.
17. **Resultados.** Reportar Recall@K; precisión de citas; groundedness evaluada; latencia; tasa de escalamiento. Comparar baseline y modelo final e interpretar los errores.
18. **Seguridad y uso responsable.** Crear análisis de amenazas, privacidad, sesgo, abuso, supervisión humana y recuperación ante fallos.
19. **Despliegue.** Preparar ambiente reproducible, datos demo, health checks y procedimiento de instalación.
20. **Cierre.** Elaborar informe, manuales, presentación, demostración y retrospectiva técnica.

13.6 Diagrama del flujo de extremo a extremo



13.7 Plan de pruebas

El equipo debe diseñar pruebas unitarias para servicios y reglas; integración para persistencia y servicio IA; pruebas de API; casos de aceptación derivados de historias de usuario; pruebas de entradas inválidas; evaluación del modelo con test independiente; y una prueba básica de rendimiento. Los defectos encontrados deben registrarse y relacionarse con la corrección aplicada.

13.8 Entregables específicos

Visión y alcance, SRS/backlog, matriz de trazabilidad, prototipos, diagramas TikZ/UML/C4, modelo de datos, repositorio Git, aplicación ejecutable, servicio IA, dataset o instrucciones reproducibles, modelo versionado, pruebas, documentación OpenAPI, Docker, manual técnico, manual de usuario, informe de métricas y video/demo.

13.9 Criterios de aceptación académica

El flujo principal debe funcionar de extremo a extremo y ser demostrable con datos de prueba. La IA debe estar realmente integrada y comparada contra un baseline. Los resultados deben ser reproducibles; los requisitos deben vincularse con pruebas; y el sistema debe manejar errores sin depender de modificaciones manuales del código durante la demostración.

14. Proyecto 12: Plataforma de administración de condominios con clasificación de solicitudes

Enfoque de IA

NLP + workflow

14.1 Descripción y objetivo

Gestionar residentes, zonas comunes, pagos operativos y solicitudes; clasificar automáticamente peticiones para enviarlas al flujo adecuado. La propuesta debe desarrollarse como un sistema de software completo. El componente inteligente debe integrarse mediante interfaces bien definidas, registrar su versión y poder evaluarse independientemente de la interfaz de usuario.

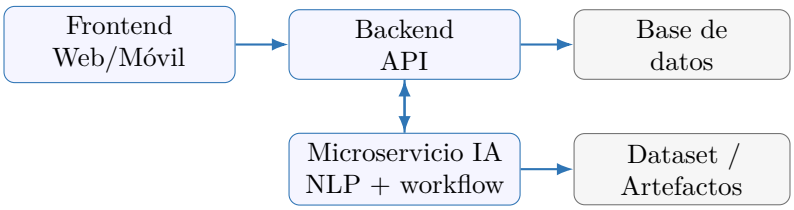
14.2 Usuarios, alcance y requisitos sugeridos

Actores sugeridos: usuario final, operador especializado y administrador. Módulos mínimos:

- Residentes
- Reservas
- Solicitudes
- Clasificación
- Workflow
- Notificaciones

Como requisitos no funcionales se deben considerar seguridad, validación, disponibilidad, manejo de errores, auditoría, rendimiento, mantenibilidad, accesibilidad, protección de datos, pruebas y documentación.

14.3 Arquitectura propuesta



El backend concentra reglas de negocio y autorización; el servicio de IA recibe entradas validadas y devuelve predicciones con metadatos. Esta separación facilita pruebas, versionado, observabilidad y reemplazo del modelo.

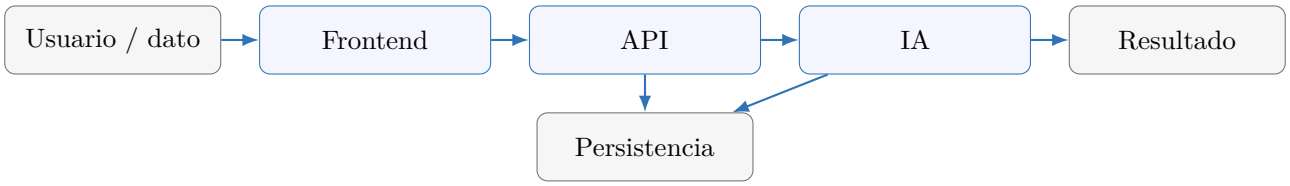
14.4 Tecnologías sugeridas

Python, FastAPI, PostgreSQL, React, scikit-learn/Transformers. El equipo puede sustituirlas justificando equivalencia funcional y arquitectónica.

14.5 Paso a paso de implementación

1. **Problema y visión.** Delimitar necesidad, usuarios, objetivos, alcance, exclusiones y métricas de éxito.
2. **Ingeniería de requisitos.** Crear épicas e historias de usuario, criterios de aceptación, requisitos no funcionales y matriz de trazabilidad.
3. **Análisis y dominio.** Identificar entidades, reglas, eventos, estados, excepciones y responsabilidades de los actores.
4. **Prototipado.** Diseñar wireframes del flujo principal y validarlos antes de programar.
5. **Arquitectura.** Crear diagramas de contexto, contenedores/componentes, secuencia y despliegue; especificar endpoints.
6. **Diseño de datos.** Definir modelo lógico, restricciones, índices, migraciones y política de auditoría.
7. **Dataset.** Documentar procedencia, licencia, variables, distribución, calidad, faltantes, sesgos y criterios de partición.
8. **EDA y baseline.** Explorar los datos y construir un modelo base sencillo que establezca una referencia cuantitativa.
9. **Entrenamiento.** Construir pipeline reproducible, seleccionar hiperparámetros usando validación y conservar un test independiente.
10. **Evaluación IA.** Medir desempeño global y por subgrupos pertinentes, analizar errores y documentar limitaciones.
11. **Backend.** Implementar lógica, persistencia, seguridad, validación, manejo de excepciones y documentación OpenAPI.
12. **Servicio IA.** Exponer inferencia mediante interfaz desacoplada y registrar versión, fecha y parámetros necesarios para trazabilidad.
13. **Frontend.** Implementar formularios, dashboards, estados de error/carga y explicación apropiada de resultados.
14. **Integración.** Ejecutar casos de extremo a extremo y verificar consistencia entre UI, API, datos y modelo.
15. **Testing.** Implementar pruebas unitarias, integración, contrato/API, sistema, aceptación y pruebas específicas para datos/modelo.
16. **DevOps/MLOps.** Configurar repositorio, CI, dependencias, variables de entorno, Docker, logging y versionado de modelos.
17. **Resultados.** Reportar Macro-F1; exactitud de enrutamiento; tiempo de atención; porcentaje de SLA cumplido. Comparar baseline y modelo final e interpretar los errores.
18. **Seguridad y uso responsable.** Crear análisis de amenazas, privacidad, sesgo, abuso, supervisión humana y recuperación ante fallos.
19. **Despliegue.** Preparar ambiente reproducible, datos demo, health checks y procedimiento de instalación.
20. **Cierre.** Elaborar informe, manuales, presentación, demostración y retrospectiva técnica.

14.6 Diagrama del flujo de extremo a extremo



14.7 Plan de pruebas

El equipo debe diseñar pruebas unitarias para servicios y reglas; integración para persistencia y servicio IA; pruebas de API; casos de aceptación derivados de historias de usuario; pruebas de entradas inválidas; evaluación del modelo con test independiente; y una prueba básica de rendimiento. Los defectos encontrados deben registrarse y relacionarse con la corrección aplicada.

14.8 Entregables específicos

Visión y alcance, SRS/backlog, matriz de trazabilidad, prototipos, diagramas TikZ/UML/C4, modelo de datos, repositorio Git, aplicación ejecutable, servicio IA, dataset o instrucciones reproducibles, modelo versionado, pruebas, documentación OpenAPI, Docker, manual técnico, manual de usuario, informe de métricas y video/demo.

14.9 Criterios de aceptación académica

El flujo principal debe funcionar de extremo a extremo y ser demostrable con datos de prueba. La IA debe estar realmente integrada y comparada contra un baseline. Los resultados deben ser reproducibles; los requisitos deben vincularse con pruebas; y el sistema debe manejar errores sin depender de modificaciones manuales del código durante la demostración.

15. Proyecto 13: Sistema inteligente de control de cadena de frío

Enfoque de IA

IoT + detección de anomalías

15.1 Descripción y objetivo

Monitorear temperatura y humedad de productos sensibles durante almacenamiento o transporte, detectando desviaciones y documentando eventos. La propuesta debe desarrollarse como un sistema de software completo. El componente inteligente debe integrarse mediante interfaces bien definidas, registrar su versión y poder evaluarse independientemente de la interfaz de usuario.

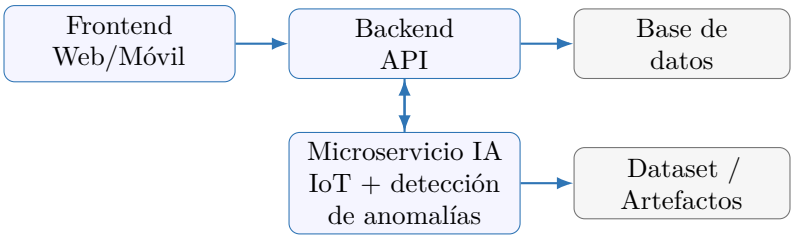
15.2 Usuarios, alcance y requisitos sugeridos

Actores sugeridos: usuario final, operador especializado y administrador. Módulos mínimos:

- Activos/lotes
- Sensores
- Telemetría
- Reglas
- Anomalías
- Trazabilidad

Como requisitos no funcionales se deben considerar seguridad, validación, disponibilidad, manejo de errores, auditoría, rendimiento, mantenibilidad, accesibilidad, protección de datos, pruebas y documentación.

15.3 Arquitectura propuesta



El backend concentra reglas de negocio y autorización; el servicio de IA recibe entradas validadas y devuelve predicciones con metadatos. Esta separación facilita pruebas, versionado, observabilidad y reemplazo del modelo.

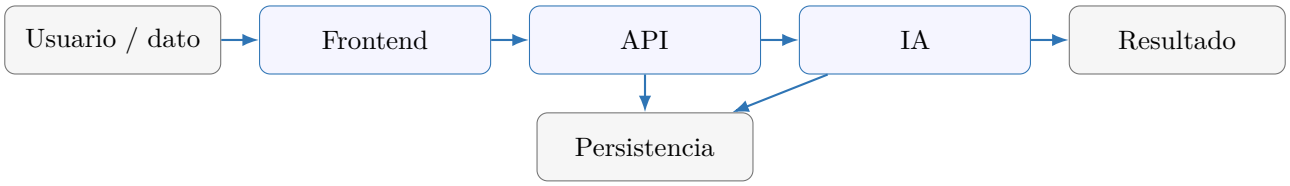
15.4 Tecnologías sugeridas

Python, FastAPI, TimescaleDB, MQTT, scikit-learn, React. El equipo puede sustituirlas justificando equivalencia funcional y arquitectónica.

15.5 Paso a paso de implementación

1. **Problema y visión.** Delimitar necesidad, usuarios, objetivos, alcance, exclusiones y métricas de éxito.
2. **Ingeniería de requisitos.** Crear épicas e historias de usuario, criterios de aceptación, requisitos no funcionales y matriz de trazabilidad.
3. **Análisis y dominio.** Identificar entidades, reglas, eventos, estados, excepciones y responsabilidades de los actores.
4. **Prototipado.** Diseñar wireframes del flujo principal y validarlos antes de programar.
5. **Arquitectura.** Crear diagramas de contexto, contenedores/componentes, secuencia y despliegue; especificar endpoints.
6. **Diseño de datos.** Definir modelo lógico, restricciones, índices, migraciones y política de auditoría.
7. **Dataset.** Documentar procedencia, licencia, variables, distribución, calidad, faltantes, sesgos y criterios de partición.
8. **EDA y baseline.** Explorar los datos y construir un modelo base sencillo que establezca una referencia cuantitativa.
9. **Entrenamiento.** Construir pipeline reproducible, seleccionar hiperparámetros usando validación y conservar un test independiente.
10. **Evaluación IA.** Medir desempeño global y por subgrupos pertinentes, analizar errores y documentar limitaciones.
11. **Backend.** Implementar lógica, persistencia, seguridad, validación, manejo de excepciones y documentación OpenAPI.
12. **Servicio IA.** Exponer inferencia mediante interfaz desacoplada y registrar versión, fecha y parámetros necesarios para trazabilidad.
13. **Frontend.** Implementar formularios, dashboards, estados de error/carga y explicación apropiada de resultados.
14. **Integración.** Ejecutar casos de extremo a extremo y verificar consistencia entre UI, API, datos y modelo.
15. **Testing.** Implementar pruebas unitarias, integración, contrato/API, sistema, aceptación y pruebas específicas para datos/modelo.
16. **DevOps/MLOps.** Configurar repositorio, CI, dependencias, variables de entorno, Docker, logging y versionado de modelos.
17. **Resultados.** Reportar Recall de excursiones; falsas alarmas; tiempo de detección; completitud de telemetría. Comparar baseline y modelo final e interpretar los errores.
18. **Seguridad y uso responsable.** Crear análisis de amenazas, privacidad, sesgo, abuso, supervisión humana y recuperación ante fallos.
19. **Despliegue.** Preparar ambiente reproducible, datos demo, health checks y procedimiento de instalación.
20. **Cierre.** Elaborar informe, manuales, presentación, demostración y retrospectiva técnica.

15.6 Diagrama del flujo de extremo a extremo



15.7 Plan de pruebas

El equipo debe diseñar pruebas unitarias para servicios y reglas; integración para persistencia y servicio IA; pruebas de API; casos de aceptación derivados de historias de usuario; pruebas de entradas inválidas; evaluación del modelo con test independiente; y una prueba básica de rendimiento. Los defectos encontrados deben registrarse y relacionarse con la corrección aplicada.

15.8 Entregables específicos

Visión y alcance, SRS/backlog, matriz de trazabilidad, prototipos, diagramas TikZ/UML/C4, modelo de datos, repositorio Git, aplicación ejecutable, servicio IA, dataset o instrucciones reproducibles, modelo versionado, pruebas, documentación OpenAPI, Docker, manual técnico, manual de usuario, informe de métricas y video/demo.

15.9 Criterios de aceptación académica

El flujo principal debe funcionar de extremo a extremo y ser demostrable con datos de prueba. La IA debe estar realmente integrada y comparada contra un baseline. Los resultados deben ser reproducibles; los requisitos deben vincularse con pruebas; y el sistema debe manejar errores sin depender de modificaciones manuales del código durante la demostración.

16. Proyecto 14: Plataforma de análisis de tráfico vehicular mediante visión artificial

Enfoque de IA

Computer Vision + series temporales

16.1 Descripción y objetivo

Procesar videos o imágenes de fuentes autorizadas para contar vehículos por categoría y analizar patrones de flujo sin identificar personas. La propuesta debe desarrollarse como un sistema de software completo. El componente inteligente debe integrarse mediante interfaces bien definidas, registrar su versión y poder evaluarse independientemente de la interfaz de usuario.

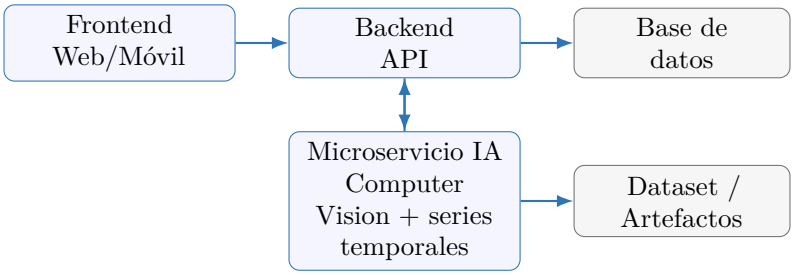
16.2 Usuarios, alcance y requisitos sugeridos

Actores sugeridos: usuario final, operador especializado y administrador. Módulos mínimos:

- Fuentes de video
- Procesamiento
- Detección/conteo
- Agregación
- Histórico
- Dashboard

Como requisitos no funcionales se deben considerar seguridad, validación, disponibilidad, manejo de errores, auditoría, rendimiento, mantenibilidad, accesibilidad, protección de datos, pruebas y documentación.

16.3 Arquitectura propuesta



El backend concentra reglas de negocio y autorización; el servicio de IA recibe entradas validadas y devuelve predicciones con metadatos. Esta separación facilita pruebas, versionado, observabilidad y reemplazo del modelo.

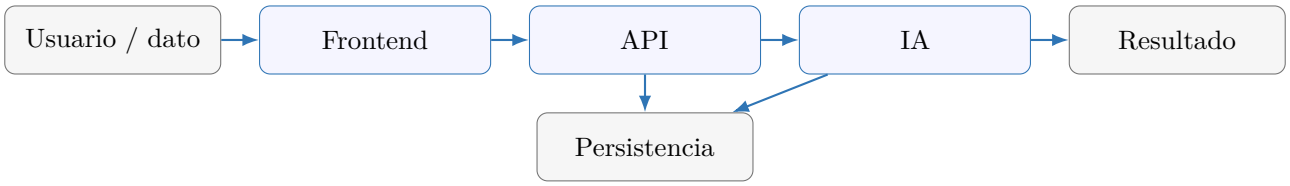
16.4 Tecnologías sugeridas

Python, OpenCV, PyTorch, FastAPI, PostgreSQL, React. El equipo puede sustituirlas justificando equivalencia funcional y arquitectónica.

16.5 Paso a paso de implementación

1. **Problema y visión.** Delimitar necesidad, usuarios, objetivos, alcance, exclusiones y métricas de éxito.
2. **Ingeniería de requisitos.** Crear épicas e historias de usuario, criterios de aceptación, requisitos no funcionales y matriz de trazabilidad.
3. **Análisis y dominio.** Identificar entidades, reglas, eventos, estados, excepciones y responsabilidades de los actores.
4. **Prototipado.** Diseñar wireframes del flujo principal y validarlos antes de programar.
5. **Arquitectura.** Crear diagramas de contexto, contenedores/componentes, secuencia y despliegue; especificar endpoints.
6. **Diseño de datos.** Definir modelo lógico, restricciones, índices, migraciones y política de auditoría.
7. **Dataset.** Documentar procedencia, licencia, variables, distribución, calidad, faltantes, sesgos y criterios de partición.
8. **EDA y baseline.** Explorar los datos y construir un modelo base sencillo que establezca una referencia cuantitativa.
9. **Entrenamiento.** Construir pipeline reproducible, seleccionar hiperparámetros usando validación y conservar un test independiente.
10. **Evaluación IA.** Medir desempeño global y por subgrupos pertinentes, analizar errores y documentar limitaciones.
11. **Backend.** Implementar lógica, persistencia, seguridad, validación, manejo de excepciones y documentación OpenAPI.
12. **Servicio IA.** Exponer inferencia mediante interfaz desacoplada y registrar versión, fecha y parámetros necesarios para trazabilidad.
13. **Frontend.** Implementar formularios, dashboards, estados de error/carga y explicación apropiada de resultados.
14. **Integración.** Ejecutar casos de extremo a extremo y verificar consistencia entre UI, API, datos y modelo.
15. **Testing.** Implementar pruebas unitarias, integración, contrato/API, sistema, aceptación y pruebas específicas para datos/modelo.
16. **DevOps/MLOps.** Configurar repositorio, CI, dependencias, variables de entorno, Docker, logging y versionado de modelos.
17. **Resultados.** Reportar mAP; error de conteo; FPS; MAE de flujo agregado; disponibilidad. Comparar baseline y modelo final e interpretar los errores.
18. **Seguridad y uso responsable.** Crear análisis de amenazas, privacidad, sesgo, abuso, supervisión humana y recuperación ante fallos.
19. **Despliegue.** Preparar ambiente reproducible, datos demo, health checks y procedimiento de instalación.
20. **Cierre.** Elaborar informe, manuales, presentación, demostración y retrospectiva técnica.

16.6 Diagrama del flujo de extremo a extremo



16.7 Plan de pruebas

El equipo debe diseñar pruebas unitarias para servicios y reglas; integración para persistencia y servicio IA; pruebas de API; casos de aceptación derivados de historias de usuario; pruebas de entradas inválidas; evaluación del modelo con test independiente; y una prueba básica de rendimiento. Los defectos encontrados deben registrarse y relacionarse con la corrección aplicada.

16.8 Entregables específicos

Visión y alcance, SRS/backlog, matriz de trazabilidad, prototipos, diagramas TikZ/UML/C4, modelo de datos, repositorio Git, aplicación ejecutable, servicio IA, dataset o instrucciones reproducibles, modelo versionado, pruebas, documentación OpenAPI, Docker, manual técnico, manual de usuario, informe de métricas y video/demo.

16.9 Criterios de aceptación académica

El flujo principal debe funcionar de extremo a extremo y ser demostrable con datos de prueba. La IA debe estar realmente integrada y comparada contra un baseline. Los resultados deben ser reproducibles; los requisitos deben vincularse con pruebas; y el sistema debe manejar errores sin depender de modificaciones manuales del código durante la demostración.

17. Proyecto 15: Sistema inteligente de clasificación de correos corporativos

Enfoque de IA

NLP + clasificación

17.1 Descripción y objetivo

Clasificar mensajes empresariales por área o tipo de solicitud y apoyar su enrutamiento, respetando permisos y privacidad. La propuesta debe desarrollarse como un sistema de software completo. El componente inteligente debe integrarse mediante interfaces bien definidas, registrar su versión y poder evaluarse independientemente de la interfaz de usuario.

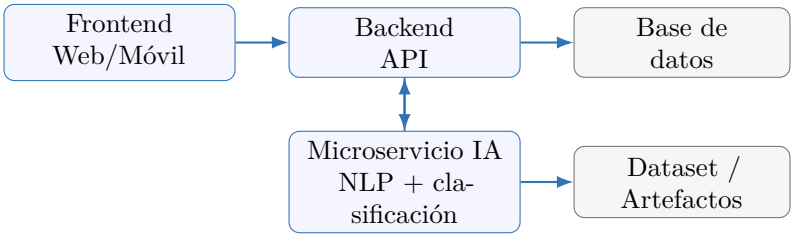
17.2 Usuarios, alcance y requisitos sugeridos

Actores sugeridos: usuario final, operador especializado y administrador. Módulos mínimos:

- Bandeja de entrada
- Preprocesamiento
- Clasificación
- Enrutamiento
- Estados
- Auditoría

Como requisitos no funcionales se deben considerar seguridad, validación, disponibilidad, manejo de errores, auditoría, rendimiento, mantenibilidad, accesibilidad, protección de datos, pruebas y documentación.

17.3 Arquitectura propuesta



El backend concentra reglas de negocio y autorización; el servicio de IA recibe entradas validadas y devuelve predicciones con metadatos. Esta separación facilita pruebas, versionado, observabilidad y reemplazo del modelo.

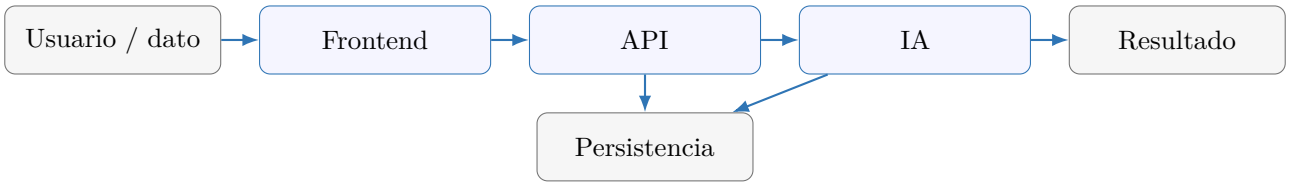
17.4 Tecnologías sugeridas

Python, FastAPI, PostgreSQL, React, Transformers/scikit-learn. El equipo puede sustituirlas justificando equivalencia funcional y arquitectónica.

17.5 Paso a paso de implementación

1. **Problema y visión.** Delimitar necesidad, usuarios, objetivos, alcance, exclusiones y métricas de éxito.
2. **Ingeniería de requisitos.** Crear épicas e historias de usuario, criterios de aceptación, requisitos no funcionales y matriz de trazabilidad.
3. **Análisis y dominio.** Identificar entidades, reglas, eventos, estados, excepciones y responsabilidades de los actores.
4. **Prototipado.** Diseñar wireframes del flujo principal y validarlos antes de programar.
5. **Arquitectura.** Crear diagramas de contexto, contenedores/componentes, secuencia y despliegue; especificar endpoints.
6. **Diseño de datos.** Definir modelo lógico, restricciones, índices, migraciones y política de auditoría.
7. **Dataset.** Documentar procedencia, licencia, variables, distribución, calidad, faltantes, sesgos y criterios de partición.
8. **EDA y baseline.** Explorar los datos y construir un modelo base sencillo que establezca una referencia cuantitativa.
9. **Entrenamiento.** Construir pipeline reproducible, seleccionar hiperparámetros usando validación y conservar un test independiente.
10. **Evaluación IA.** Medir desempeño global y por subgrupos pertinentes, analizar errores y documentar limitaciones.
11. **Backend.** Implementar lógica, persistencia, seguridad, validación, manejo de excepciones y documentación OpenAPI.
12. **Servicio IA.** Exponer inferencia mediante interfaz desacoplada y registrar versión, fecha y parámetros necesarios para trazabilidad.
13. **Frontend.** Implementar formularios, dashboards, estados de error/carga y explicación apropiada de resultados.
14. **Integración.** Ejecutar casos de extremo a extremo y verificar consistencia entre UI, API, datos y modelo.
15. **Testing.** Implementar pruebas unitarias, integración, contrato/API, sistema, aceptación y pruebas específicas para datos/modelo.
16. **DevOps/MLOps.** Configurar repositorio, CI, dependencias, variables de entorno, Docker, logging y versionado de modelos.
17. **Resultados.** Reportar Macro-F1; precisión de enrutamiento; tiempo de clasificación; correcciones humanas. Comparar baseline y modelo final e interpretar los errores.
18. **Seguridad y uso responsable.** Crear análisis de amenazas, privacidad, sesgo, abuso, supervisión humana y recuperación ante fallos.
19. **Despliegue.** Preparar ambiente reproducible, datos demo, health checks y procedimiento de instalación.
20. **Cierre.** Elaborar informe, manuales, presentación, demostración y retrospectiva técnica.

17.6 Diagrama del flujo de extremo a extremo



17.7 Plan de pruebas

El equipo debe diseñar pruebas unitarias para servicios y reglas; integración para persistencia y servicio IA; pruebas de API; casos de aceptación derivados de historias de usuario; pruebas de entradas inválidas; evaluación del modelo con test independiente; y una prueba básica de rendimiento. Los defectos encontrados deben registrarse y relacionarse con la corrección aplicada.

17.8 Entregables específicos

Visión y alcance, SRS/backlog, matriz de trazabilidad, prototipos, diagramas TikZ/UML/C4, modelo de datos, repositorio Git, aplicación ejecutable, servicio IA, dataset o instrucciones reproducibles, modelo versionado, pruebas, documentación OpenAPI, Docker, manual técnico, manual de usuario, informe de métricas y video/demo.

17.9 Criterios de aceptación académica

El flujo principal debe funcionar de extremo a extremo y ser demostrable con datos de prueba. La IA debe estar realmente integrada y comparada contra un baseline. Los resultados deben ser reproducibles; los requisitos deben vincularse con pruebas; y el sistema debe manejar errores sin depender de modificaciones manuales del código durante la demostración.

18. Proyecto 16: Plataforma de gestión de laboratorios con predicción de utilización

Enfoque de IA

Forecasting + optimización

18.1 Descripción y objetivo

Administrar laboratorios, equipos, horarios y reservas y anticipar demanda para mejorar asignación de recursos. La propuesta debe desarrollarse como un sistema de software completo. El componente inteligente debe integrarse mediante interfaces bien definidas, registrar su versión y poder evaluarse independientemente de la interfaz de usuario.

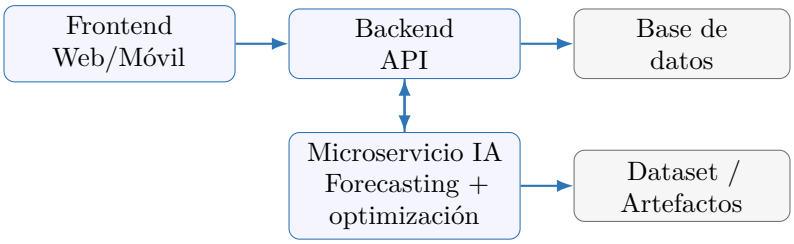
18.2 Usuarios, alcance y requisitos sugeridos

Actores sugeridos: usuario final, operador especializado y administrador. Módulos mínimos:

- Laboratorios
- Equipos
- Horarios
- Reservas
- Pronóstico
- Planificador

Como requisitos no funcionales se deben considerar seguridad, validación, disponibilidad, manejo de errores, auditoría, rendimiento, mantenibilidad, accesibilidad, protección de datos, pruebas y documentación.

18.3 Arquitectura propuesta



El backend concentra reglas de negocio y autorización; el servicio de IA recibe entradas validadas y devuelve predicciones con metadatos. Esta separación facilita pruebas, versionado, observabilidad y reemplazo del modelo.

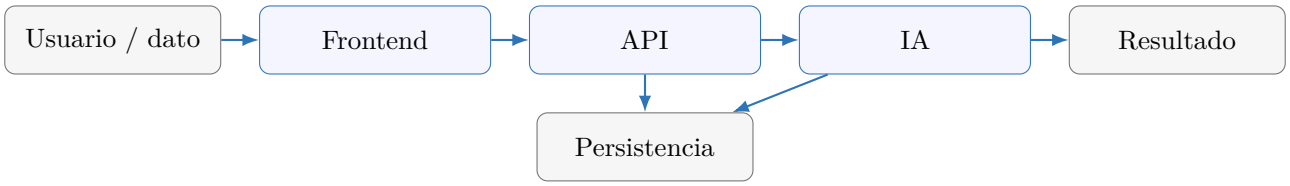
18.4 Tecnologías sugeridas

Python, FastAPI, PostgreSQL, React, scikit-learn, OR-Tools. El equipo puede sustituirlas justificando equivalencia funcional y arquitectónica.

18.5 Paso a paso de implementación

1. **Problema y visión.** Delimitar necesidad, usuarios, objetivos, alcance, exclusiones y métricas de éxito.
2. **Ingeniería de requisitos.** Crear épicas e historias de usuario, criterios de aceptación, requisitos no funcionales y matriz de trazabilidad.
3. **Análisis y dominio.** Identificar entidades, reglas, eventos, estados, excepciones y responsabilidades de los actores.
4. **Prototipado.** Diseñar wireframes del flujo principal y validarlos antes de programar.
5. **Arquitectura.** Crear diagramas de contexto, contenedores/componentes, secuencia y despliegue; especificar endpoints.
6. **Diseño de datos.** Definir modelo lógico, restricciones, índices, migraciones y política de auditoría.
7. **Dataset.** Documentar procedencia, licencia, variables, distribución, calidad, faltantes, sesgos y criterios de partición.
8. **EDA y baseline.** Explorar los datos y construir un modelo base sencillo que establezca una referencia cuantitativa.
9. **Entrenamiento.** Construir pipeline reproducible, seleccionar hiperparámetros usando validación y conservar un test independiente.
10. **Evaluación IA.** Medir desempeño global y por subgrupos pertinentes, analizar errores y documentar limitaciones.
11. **Backend.** Implementar lógica, persistencia, seguridad, validación, manejo de excepciones y documentación OpenAPI.
12. **Servicio IA.** Exponer inferencia mediante interfaz desacoplada y registrar versión, fecha y parámetros necesarios para trazabilidad.
13. **Frontend.** Implementar formularios, dashboards, estados de error/carga y explicación apropiada de resultados.
14. **Integración.** Ejecutar casos de extremo a extremo y verificar consistencia entre UI, API, datos y modelo.
15. **Testing.** Implementar pruebas unitarias, integración, contrato/API, sistema, aceptación y pruebas específicas para datos/modelo.
16. **DevOps/MLOps.** Configurar repositorio, CI, dependencias, variables de entorno, Docker, logging y versionado de modelos.
17. **Resultados.** Reportar MAE de demanda; utilización; conflictos; capacidad ociosa; tiempo de planificación. Comparar baseline y modelo final e interpretar los errores.
18. **Seguridad y uso responsable.** Crear análisis de amenazas, privacidad, sesgo, abuso, supervisión humana y recuperación ante fallos.
19. **Despliegue.** Preparar ambiente reproducible, datos demo, health checks y procedimiento de instalación.
20. **Cierre.** Elaborar informe, manuales, presentación, demostración y retrospectiva técnica.

18.6 Diagrama del flujo de extremo a extremo



18.7 Plan de pruebas

El equipo debe diseñar pruebas unitarias para servicios y reglas; integración para persistencia y servicio IA; pruebas de API; casos de aceptación derivados de historias de usuario; pruebas de entradas inválidas; evaluación del modelo con test independiente; y una prueba básica de rendimiento. Los defectos encontrados deben registrarse y relacionarse con la corrección aplicada.

18.8 Entregables específicos

Visión y alcance, SRS/backlog, matriz de trazabilidad, prototipos, diagramas TikZ/UML/C4, modelo de datos, repositorio Git, aplicación ejecutable, servicio IA, dataset o instrucciones reproducibles, modelo versionado, pruebas, documentación OpenAPI, Docker, manual técnico, manual de usuario, informe de métricas y video/demo.

18.9 Criterios de aceptación académica

El flujo principal debe funcionar de extremo a extremo y ser demostrable con datos de prueba. La IA debe estar realmente integrada y comparada contra un baseline. Los resultados deben ser reproducibles; los requisitos deben vincularse con pruebas; y el sistema debe manejar errores sin depender de modificaciones manuales del código durante la demostración.

19. Proyecto 17: Sistema inteligente de inspección visual de productos agrícolas

Enfoque de IA

Computer Vision

19.1 Descripción y objetivo

Clasificar calidad visual de frutas, granos u otros productos a partir de imágenes, registrando lote, revisión humana y trazabilidad. La propuesta debe desarrollarse como un sistema de software completo. El componente inteligente debe integrarse mediante interfaces bien definidas, registrar su versión y poder evaluarse independientemente de la interfaz de usuario.

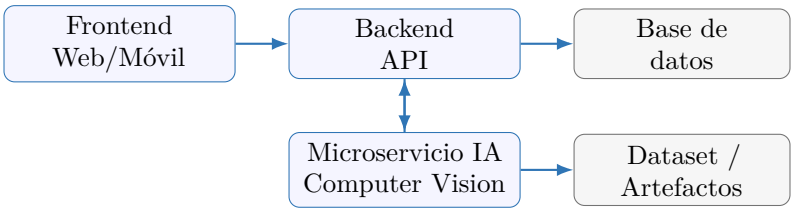
19.2 Usuarios, alcance y requisitos sugeridos

Actores sugeridos: usuario final, operador especializado y administrador. Módulos mínimos:

- Lotes
- Captura
- Preprocesamiento
- Clasificación
- Revisión
- Reportes

Como requisitos no funcionales se deben considerar seguridad, validación, disponibilidad, manejo de errores, auditoría, rendimiento, mantenibilidad, accesibilidad, protección de datos, pruebas y documentación.

19.3 Arquitectura propuesta



El backend concentra reglas de negocio y autorización; el servicio de IA recibe entradas validadas y devuelve predicciones con metadatos. Esta separación facilita pruebas, versionado, observabilidad y reemplazo del modelo.

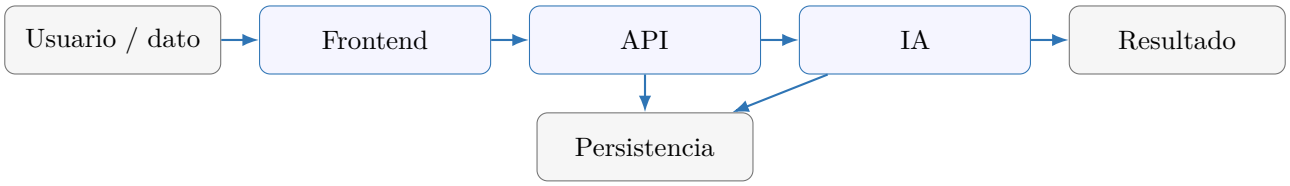
19.4 Tecnologías sugeridas

Python, PyTorch/TensorFlow, OpenCV, FastAPI, PostgreSQL, React/Gradio. El equipo puede sustituirlas justificando equivalencia funcional y arquitectónica.

19.5 Paso a paso de implementación

1. **Problema y visión.** Delimitar necesidad, usuarios, objetivos, alcance, exclusiones y métricas de éxito.
2. **Ingeniería de requisitos.** Crear épicas e historias de usuario, criterios de aceptación, requisitos no funcionales y matriz de trazabilidad.
3. **Análisis y dominio.** Identificar entidades, reglas, eventos, estados, excepciones y responsabilidades de los actores.
4. **Prototipado.** Diseñar wireframes del flujo principal y validarlos antes de programar.
5. **Arquitectura.** Crear diagramas de contexto, contenedores/componentes, secuencia y despliegue; especificar endpoints.
6. **Diseño de datos.** Definir modelo lógico, restricciones, índices, migraciones y política de auditoría.
7. **Dataset.** Documentar procedencia, licencia, variables, distribución, calidad, faltantes, sesgos y criterios de partición.
8. **EDA y baseline.** Explorar los datos y construir un modelo base sencillo que establezca una referencia cuantitativa.
9. **Entrenamiento.** Construir pipeline reproducible, seleccionar hiperparámetros usando validación y conservar un test independiente.
10. **Evaluación IA.** Medir desempeño global y por subgrupos pertinentes, analizar errores y documentar limitaciones.
11. **Backend.** Implementar lógica, persistencia, seguridad, validación, manejo de excepciones y documentación OpenAPI.
12. **Servicio IA.** Exponer inferencia mediante interfaz desacoplada y registrar versión, fecha y parámetros necesarios para trazabilidad.
13. **Frontend.** Implementar formularios, dashboards, estados de error/carga y explicación apropiada de resultados.
14. **Integración.** Ejecutar casos de extremo a extremo y verificar consistencia entre UI, API, datos y modelo.
15. **Testing.** Implementar pruebas unitarias, integración, contrato/API, sistema, aceptación y pruebas específicas para datos/modelo.
16. **DevOps/MLOps.** Configurar repositorio, CI, dependencias, variables de entorno, Docker, logging y versionado de modelos.
17. **Resultados.** Reportar Macro-F1; precision/recall por categoría; latencia; acuerdo con revisión humana. Comparar baseline y modelo final e interpretar los errores.
18. **Seguridad y uso responsable.** Crear análisis de amenazas, privacidad, sesgo, abuso, supervisión humana y recuperación ante fallos.
19. **Despliegue.** Preparar ambiente reproducible, datos demo, health checks y procedimiento de instalación.
20. **Cierre.** Elaborar informe, manuales, presentación, demostración y retrospectiva técnica.

19.6 Diagrama del flujo de extremo a extremo



19.7 Plan de pruebas

El equipo debe diseñar pruebas unitarias para servicios y reglas; integración para persistencia y servicio IA; pruebas de API; casos de aceptación derivados de historias de usuario; pruebas de entradas inválidas; evaluación del modelo con test independiente; y una prueba básica de rendimiento. Los defectos encontrados deben registrarse y relacionarse con la corrección aplicada.

19.8 Entregables específicos

Visión y alcance, SRS/backlog, matriz de trazabilidad, prototipos, diagramas TikZ/UML/C4, modelo de datos, repositorio Git, aplicación ejecutable, servicio IA, dataset o instrucciones reproducibles, modelo versionado, pruebas, documentación OpenAPI, Docker, manual técnico, manual de usuario, informe de métricas y video/demo.

19.9 Criterios de aceptación académica

El flujo principal debe funcionar de extremo a extremo y ser demostrable con datos de prueba. La IA debe estar realmente integrada y comparada contra un baseline. Los resultados deben ser reproducibles; los requisitos deben vincularse con pruebas; y el sistema debe manejar errores sin depender de modificaciones manuales del código durante la demostración.

20. Proyecto 18: Plataforma de gestión de voluntariado con asignación inteligente

Enfoque de IA

Matching/recomendación

20.1 Descripción y objetivo

Gestionar oportunidades, disponibilidad, habilidades declaradas y participación y sugerir coincidencias compatibles con los requisitos de cada actividad. La propuesta debe desarrollarse como un sistema de software completo. El componente inteligente debe integrarse mediante interfaces bien definidas, registrar su versión y poder evaluarse independientemente de la interfaz de usuario.

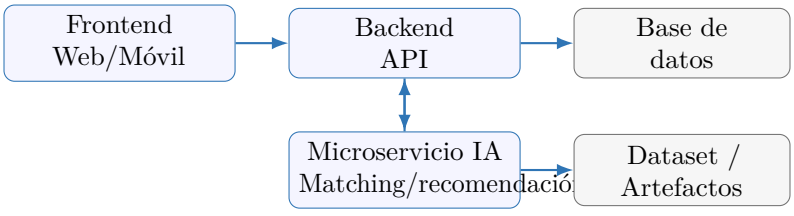
20.2 Usuarios, alcance y requisitos sugeridos

Actores sugeridos: usuario final, operador especializado y administrador. Módulos mínimos:

- Voluntarios
- Organizaciones
- Actividades
- Disponibilidad
- Matching
- Seguimiento

Como requisitos no funcionales se deben considerar seguridad, validación, disponibilidad, manejo de errores, auditoría, rendimiento, mantenibilidad, accesibilidad, protección de datos, pruebas y documentación.

20.3 Arquitectura propuesta



El backend concentra reglas de negocio y autorización; el servicio de IA recibe entradas validadas y devuelve predicciones con metadatos. Esta separación facilita pruebas, versionado, observabilidad y reemplazo del modelo.

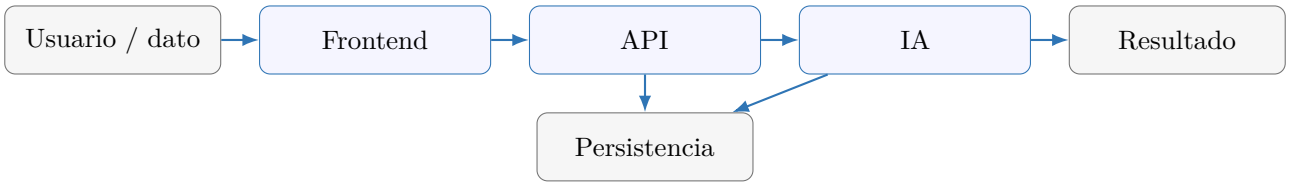
20.4 Tecnologías sugeridas

Python, FastAPI, PostgreSQL, React, scikit-learn/OR-Tools. El equipo puede sustituirlas justificando equivalencia funcional y arquitectónica.

20.5 Paso a paso de implementación

1. **Problema y visión.** Delimitar necesidad, usuarios, objetivos, alcance, exclusiones y métricas de éxito.
2. **Ingeniería de requisitos.** Crear épicas e historias de usuario, criterios de aceptación, requisitos no funcionales y matriz de trazabilidad.
3. **Análisis y dominio.** Identificar entidades, reglas, eventos, estados, excepciones y responsabilidades de los actores.
4. **Prototipado.** Diseñar wireframes del flujo principal y validarlos antes de programar.
5. **Arquitectura.** Crear diagramas de contexto, contenedores/componentes, secuencia y despliegue; especificar endpoints.
6. **Diseño de datos.** Definir modelo lógico, restricciones, índices, migraciones y política de auditoría.
7. **Dataset.** Documentar procedencia, licencia, variables, distribución, calidad, faltantes, sesgos y criterios de partición.
8. **EDA y baseline.** Explorar los datos y construir un modelo base sencillo que establezca una referencia cuantitativa.
9. **Entrenamiento.** Construir pipeline reproducible, seleccionar hiperparámetros usando validación y conservar un test independiente.
10. **Evaluación IA.** Medir desempeño global y por subgrupos pertinentes, analizar errores y documentar limitaciones.
11. **Backend.** Implementar lógica, persistencia, seguridad, validación, manejo de excepciones y documentación OpenAPI.
12. **Servicio IA.** Exponer inferencia mediante interfaz desacoplada y registrar versión, fecha y parámetros necesarios para trazabilidad.
13. **Frontend.** Implementar formularios, dashboards, estados de error/carga y explicación apropiada de resultados.
14. **Integración.** Ejecutar casos de extremo a extremo y verificar consistencia entre UI, API, datos y modelo.
15. **Testing.** Implementar pruebas unitarias, integración, contrato/API, sistema, aceptación y pruebas específicas para datos/modelo.
16. **DevOps/MLOps.** Configurar repositorio, CI, dependencias, variables de entorno, Docker, logging y versionado de modelos.
17. **Resultados.** Reportar Precision@K; cobertura de vacantes; porcentaje de coincidencias aceptadas; restricciones satisfechas. Comparar baseline y modelo final e interpretar los errores.
18. **Seguridad y uso responsable.** Crear análisis de amenazas, privacidad, sesgo, abuso, supervisión humana y recuperación ante fallos.
19. **Despliegue.** Preparar ambiente reproducible, datos demo, health checks y procedimiento de instalación.
20. **Cierre.** Elaborar informe, manuales, presentación, demostración y retrospectiva técnica.

20.6 Diagrama del flujo de extremo a extremo



20.7 Plan de pruebas

El equipo debe diseñar pruebas unitarias para servicios y reglas; integración para persistencia y servicio IA; pruebas de API; casos de aceptación derivados de historias de usuario; pruebas de entradas inválidas; evaluación del modelo con test independiente; y una prueba básica de rendimiento. Los defectos encontrados deben registrarse y relacionarse con la corrección aplicada.

20.8 Entregables específicos

Visión y alcance, SRS/backlog, matriz de trazabilidad, prototipos, diagramas TikZ/UML/C4, modelo de datos, repositorio Git, aplicación ejecutable, servicio IA, dataset o instrucciones reproducibles, modelo versionado, pruebas, documentación OpenAPI, Docker, manual técnico, manual de usuario, informe de métricas y video/demo.

20.9 Criterios de aceptación académica

El flujo principal debe funcionar de extremo a extremo y ser demostrable con datos de prueba. La IA debe estar realmente integrada y comparada contra un baseline. Los resultados deben ser reproducibles; los requisitos deben vincularse con pruebas; y el sistema debe manejar errores sin depender de modificaciones manuales del código durante la demostración.

21. Proyecto 19: Sistema inteligente de seguimiento de proyectos de construcción

Enfoque de IA

Visión + analítica de avance

21.1 Descripción y objetivo

Gestionar actividades, hitos y evidencias fotográficas y estimar indicadores de avance visual para compararlos con el cronograma bajo revisión del responsable. La propuesta debe desarrollarse como un sistema de software completo. El componente inteligente debe integrarse mediante interfaces bien definidas, registrar su versión y poder evaluarse independientemente de la interfaz de usuario.

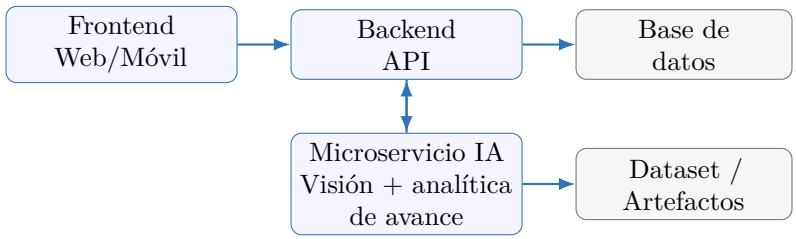
21.2 Usuarios, alcance y requisitos sugeridos

Actores sugeridos: usuario final, operador especializado y administrador. Módulos mínimos:

- Proyectos
- WBS
- Cronograma
- Evidencias
- Análisis visual
- Dashboard

Como requisitos no funcionales se deben considerar seguridad, validación, disponibilidad, manejo de errores, auditoría, rendimiento, mantenibilidad, accesibilidad, protección de datos, pruebas y documentación.

21.3 Arquitectura propuesta



El backend concentra reglas de negocio y autorización; el servicio de IA recibe entradas validadas y devuelve predicciones con metadatos. Esta separación facilita pruebas, versionado, observabilidad y reemplazo del modelo.

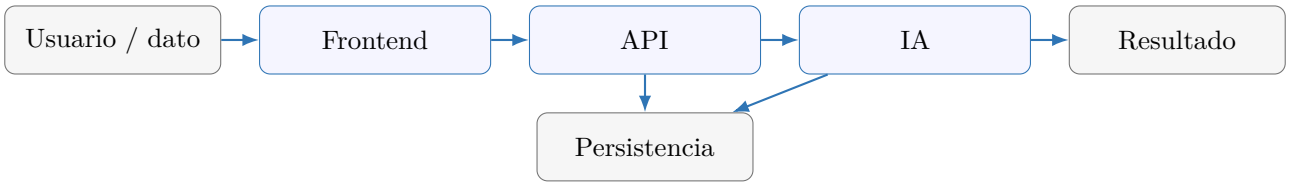
21.4 Tecnologías sugeridas

Python, FastAPI, PostgreSQL, React, PyTorch/OpenCV, Docker. El equipo puede sustituirlas justificando equivalencia funcional y arquitectónica.

21.5 Paso a paso de implementación

1. **Problema y visión.** Delimitar necesidad, usuarios, objetivos, alcance, exclusiones y métricas de éxito.
2. **Ingeniería de requisitos.** Crear épicas e historias de usuario, criterios de aceptación, requisitos no funcionales y matriz de trazabilidad.
3. **Análisis y dominio.** Identificar entidades, reglas, eventos, estados, excepciones y responsabilidades de los actores.
4. **Prototipado.** Diseñar wireframes del flujo principal y validarlos antes de programar.
5. **Arquitectura.** Crear diagramas de contexto, contenedores/componentes, secuencia y despliegue; especificar endpoints.
6. **Diseño de datos.** Definir modelo lógico, restricciones, índices, migraciones y política de auditoría.
7. **Dataset.** Documentar procedencia, licencia, variables, distribución, calidad, faltantes, sesgos y criterios de partición.
8. **EDA y baseline.** Explorar los datos y construir un modelo base sencillo que establezca una referencia cuantitativa.
9. **Entrenamiento.** Construir pipeline reproducible, seleccionar hiperparámetros usando validación y conservar un test independiente.
10. **Evaluación IA.** Medir desempeño global y por subgrupos pertinentes, analizar errores y documentar limitaciones.
11. **Backend.** Implementar lógica, persistencia, seguridad, validación, manejo de excepciones y documentación OpenAPI.
12. **Servicio IA.** Exponer inferencia mediante interfaz desacoplada y registrar versión, fecha y parámetros necesarios para trazabilidad.
13. **Frontend.** Implementar formularios, dashboards, estados de error/carga y explicación apropiada de resultados.
14. **Integración.** Ejecutar casos de extremo a extremo y verificar consistencia entre UI, API, datos y modelo.
15. **Testing.** Implementar pruebas unitarias, integración, contrato/API, sistema, aceptación y pruebas específicas para datos/modelo.
16. **DevOps/MLOps.** Configurar repositorio, CI, dependencias, variables de entorno, Docker, logging y versionado de modelos.
17. **Resultados.** Reportar Error de estimación de avance; cumplimiento de hitos; precision de hallazgos; tiempo de procesamiento. Comparar baseline y modelo final e interpretar los errores.
18. **Seguridad y uso responsable.** Crear análisis de amenazas, privacidad, sesgo, abuso, supervisión humana y recuperación ante fallos.
19. **Despliegue.** Preparar ambiente reproducible, datos demo, health checks y procedimiento de instalación.
20. **Cierre.** Elaborar informe, manuales, presentación, demostración y retrospectiva técnica.

21.6 Diagrama del flujo de extremo a extremo



21.7 Plan de pruebas

El equipo debe diseñar pruebas unitarias para servicios y reglas; integración para persistencia y servicio IA; pruebas de API; casos de aceptación derivados de historias de usuario; pruebas de entradas inválidas; evaluación del modelo con test independiente; y una prueba básica de rendimiento. Los defectos encontrados deben registrarse y relacionarse con la corrección aplicada.

21.8 Entregables específicos

Visión y alcance, SRS/backlog, matriz de trazabilidad, prototipos, diagramas TikZ/UML/C4, modelo de datos, repositorio Git, aplicación ejecutable, servicio IA, dataset o instrucciones reproducibles, modelo versionado, pruebas, documentación OpenAPI, Docker, manual técnico, manual de usuario, informe de métricas y video/demo.

21.9 Criterios de aceptación académica

El flujo principal debe funcionar de extremo a extremo y ser demostrable con datos de prueba. La IA debe estar realmente integrada y comparada contra un baseline. Los resultados deben ser reproducibles; los requisitos deben vincularse con pruebas; y el sistema debe manejar errores sin depender de modificaciones manuales del código durante la demostración.

22. Proyecto 20: Plataforma de gestión de centros de cómputo con predicción de capacidad

Enfoque de IA

Forecasting + anomalías

22.1 Descripción y objetivo

Registrar uso de CPU, memoria, almacenamiento y trabajos para anticipar necesidades de capacidad y detectar patrones atípicos. La propuesta debe desarrollarse como un sistema de software completo. El componente inteligente debe integrarse mediante interfaces bien definidas, registrar su versión y poder evaluarse independientemente de la interfaz de usuario.

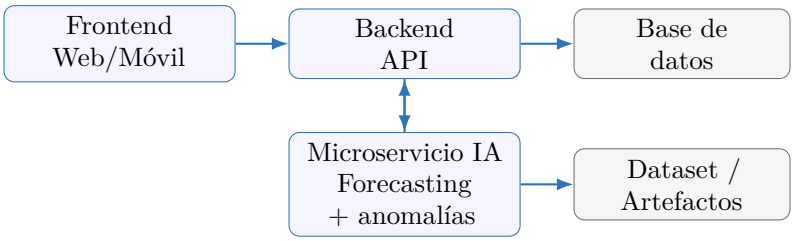
22.2 Usuarios, alcance y requisitos sugeridos

Actores sugeridos: usuario final, operador especializado y administrador. Módulos mínimos:

- Servidores
- Telemetría
- Jobs
- Histórico
- Forecast
- Alertas de capacidad

Como requisitos no funcionales se deben considerar seguridad, validación, disponibilidad, manejo de errores, auditoría, rendimiento, mantenibilidad, accesibilidad, protección de datos, pruebas y documentación.

22.3 Arquitectura propuesta



El backend concentra reglas de negocio y autorización; el servicio de IA recibe entradas validadas y devuelve predicciones con metadatos. Esta separación facilita pruebas, versionado, observabilidad y reemplazo del modelo.

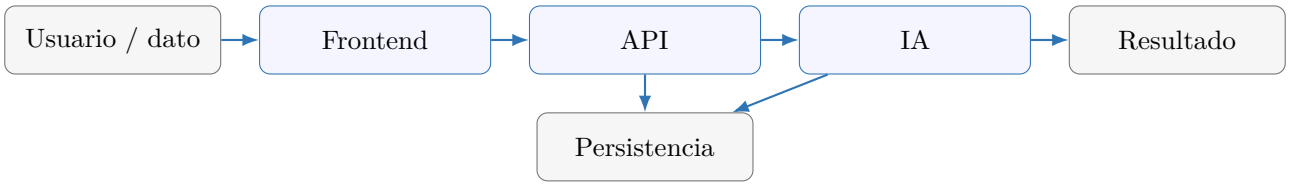
22.4 Tecnologías sugeridas

Python, FastAPI, TimescaleDB, Prometheus, scikit-learn, Grafana/React. El equipo puede sustituirlas justificando equivalencia funcional y arquitectónica.

22.5 Paso a paso de implementación

1. **Problema y visión.** Delimitar necesidad, usuarios, objetivos, alcance, exclusiones y métricas de éxito.
2. **Ingeniería de requisitos.** Crear épicas e historias de usuario, criterios de aceptación, requisitos no funcionales y matriz de trazabilidad.
3. **Análisis y dominio.** Identificar entidades, reglas, eventos, estados, excepciones y responsabilidades de los actores.
4. **Prototipado.** Diseñar wireframes del flujo principal y validarlos antes de programar.
5. **Arquitectura.** Crear diagramas de contexto, contenedores/componentes, secuencia y despliegue; especificar endpoints.
6. **Diseño de datos.** Definir modelo lógico, restricciones, índices, migraciones y política de auditoría.
7. **Dataset.** Documentar procedencia, licencia, variables, distribución, calidad, faltantes, sesgos y criterios de partición.
8. **EDA y baseline.** Explorar los datos y construir un modelo base sencillo que establezca una referencia cuantitativa.
9. **Entrenamiento.** Construir pipeline reproducible, seleccionar hiperparámetros usando validación y conservar un test independiente.
10. **Evaluación IA.** Medir desempeño global y por subgrupos pertinentes, analizar errores y documentar limitaciones.
11. **Backend.** Implementar lógica, persistencia, seguridad, validación, manejo de excepciones y documentación OpenAPI.
12. **Servicio IA.** Exponer inferencia mediante interfaz desacoplada y registrar versión, fecha y parámetros necesarios para trazabilidad.
13. **Frontend.** Implementar formularios, dashboards, estados de error/carga y explicación apropiada de resultados.
14. **Integración.** Ejecutar casos de extremo a extremo y verificar consistencia entre UI, API, datos y modelo.
15. **Testing.** Implementar pruebas unitarias, integración, contrato/API, sistema, aceptación y pruebas específicas para datos/modelo.
16. **DevOps/MLOps.** Configurar repositorio, CI, dependencias, variables de entorno, Docker, logging y versionado de modelos.
17. **Resultados.** Reportar MAE/sMAPE de recursos; recall de saturaciones; falsas alertas; disponibilidad. Comparar baseline y modelo final e interpretar los errores.
18. **Seguridad y uso responsable.** Crear análisis de amenazas, privacidad, sesgo, abuso, supervisión humana y recuperación ante fallos.
19. **Despliegue.** Preparar ambiente reproducible, datos demo, health checks y procedimiento de instalación.
20. **Cierre.** Elaborar informe, manuales, presentación, demostración y retrospectiva técnica.

22.6 Diagrama del flujo de extremo a extremo



22.7 Plan de pruebas

El equipo debe diseñar pruebas unitarias para servicios y reglas; integración para persistencia y servicio IA; pruebas de API; casos de aceptación derivados de historias de usuario; pruebas de entradas inválidas; evaluación del modelo con test independiente; y una prueba básica de rendimiento. Los defectos encontrados deben registrarse y relacionarse con la corrección aplicada.

22.8 Entregables específicos

Visión y alcance, SRS/backlog, matriz de trazabilidad, prototipos, diagramas TikZ/UML/C4, modelo de datos, repositorio Git, aplicación ejecutable, servicio IA, dataset o instrucciones reproducibles, modelo versionado, pruebas, documentación OpenAPI, Docker, manual técnico, manual de usuario, informe de métricas y video/demo.

22.9 Criterios de aceptación académica

El flujo principal debe funcionar de extremo a extremo y ser demostrable con datos de prueba. La IA debe estar realmente integrada y comparada contra un baseline. Los resultados deben ser reproducibles; los requisitos deben vincularse con pruebas; y el sistema debe manejar errores sin depender de modificaciones manuales del código durante la demostración.

23. Proyecto 21: Sistema de clasificación automática de noticias por temática para hemeroteca

Enfoque de IA

NLP + clasificación temática

23.1 Descripción y objetivo

Gestionar un repositorio académico de noticias y clasificarlas por categorías temáticas, entidades y palabras clave para facilitar consulta; no realiza valoración política ni persuasión. La propuesta debe desarrollarse como un sistema de software completo. El componente inteligente debe integrarse mediante interfaces bien definidas, registrar su versión y poder evaluarse independientemente de la interfaz de usuario.

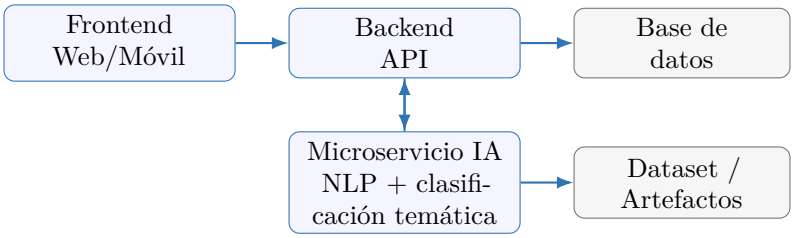
23.2 Usuarios, alcance y requisitos sugeridos

Actores sugeridos: usuario final, operador especializado y administrador. Módulos mínimos:

- Fuentes autorizadas
- Ingesta
- Metadatos
- Clasificación
- Entidades
- Buscador

Como requisitos no funcionales se deben considerar seguridad, validación, disponibilidad, manejo de errores, auditoría, rendimiento, mantenibilidad, accesibilidad, protección de datos, pruebas y documentación.

23.3 Arquitectura propuesta



El backend concentra reglas de negocio y autorización; el servicio de IA recibe entradas validadas y devuelve predicciones con metadatos. Esta separación facilita pruebas, versionado, observabilidad y reemplazo del modelo.

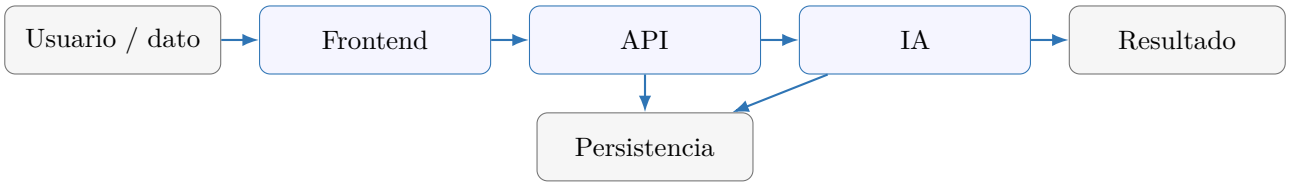
23.4 Tecnologías sugeridas

Python, FastAPI, PostgreSQL, Elasticsearch, Transformers, React. El equipo puede sustituirlas justificando equivalencia funcional y arquitectónica.

23.5 Paso a paso de implementación

1. **Problema y visión.** Delimitar necesidad, usuarios, objetivos, alcance, exclusiones y métricas de éxito.
2. **Ingeniería de requisitos.** Crear épicas e historias de usuario, criterios de aceptación, requisitos no funcionales y matriz de trazabilidad.
3. **Análisis y dominio.** Identificar entidades, reglas, eventos, estados, excepciones y responsabilidades de los actores.
4. **Prototipado.** Diseñar wireframes del flujo principal y validarlos antes de programar.
5. **Arquitectura.** Crear diagramas de contexto, contenedores/componentes, secuencia y despliegue; especificar endpoints.
6. **Diseño de datos.** Definir modelo lógico, restricciones, índices, migraciones y política de auditoría.
7. **Dataset.** Documentar procedencia, licencia, variables, distribución, calidad, faltantes, sesgos y criterios de partición.
8. **EDA y baseline.** Explorar los datos y construir un modelo base sencillo que establezca una referencia cuantitativa.
9. **Entrenamiento.** Construir pipeline reproducible, seleccionar hiperparámetros usando validación y conservar un test independiente.
10. **Evaluación IA.** Medir desempeño global y por subgrupos pertinentes, analizar errores y documentar limitaciones.
11. **Backend.** Implementar lógica, persistencia, seguridad, validación, manejo de excepciones y documentación OpenAPI.
12. **Servicio IA.** Exponer inferencia mediante interfaz desacoplada y registrar versión, fecha y parámetros necesarios para trazabilidad.
13. **Frontend.** Implementar formularios, dashboards, estados de error/carga y explicación apropiada de resultados.
14. **Integración.** Ejecutar casos de extremo a extremo y verificar consistencia entre UI, API, datos y modelo.
15. **Testing.** Implementar pruebas unitarias, integración, contrato/API, sistema, aceptación y pruebas específicas para datos/modelo.
16. **DevOps/MLOps.** Configurar repositorio, CI, dependencias, variables de entorno, Docker, logging y versionado de modelos.
17. **Resultados.** Reportar Macro-F1; precisión de entidades; Recall@K de búsqueda; latencia de indexación. Comparar baseline y modelo final e interpretar los errores.
18. **Seguridad y uso responsable.** Crear análisis de amenazas, privacidad, sesgo, abuso, supervisión humana y recuperación ante fallos.
19. **Despliegue.** Preparar ambiente reproducible, datos demo, health checks y procedimiento de instalación.
20. **Cierre.** Elaborar informe, manuales, presentación, demostración y retrospectiva técnica.

23.6 Diagrama del flujo de extremo a extremo



23.7 Plan de pruebas

El equipo debe diseñar pruebas unitarias para servicios y reglas; integración para persistencia y servicio IA; pruebas de API; casos de aceptación derivados de historias de usuario; pruebas de entradas inválidas; evaluación del modelo con test independiente; y una prueba básica de rendimiento. Los defectos encontrados deben registrarse y relacionarse con la corrección aplicada.

23.8 Entregables específicos

Visión y alcance, SRS/backlog, matriz de trazabilidad, prototipos, diagramas TikZ/UML/C4, modelo de datos, repositorio Git, aplicación ejecutable, servicio IA, dataset o instrucciones reproducibles, modelo versionado, pruebas, documentación OpenAPI, Docker, manual técnico, manual de usuario, informe de métricas y video/demo.

23.9 Criterios de aceptación académica

El flujo principal debe funcionar de extremo a extremo y ser demostrable con datos de prueba. La IA debe estar realmente integrada y comparada contra un baseline. Los resultados deben ser reproducibles; los requisitos deben vincularse con pruebas; y el sistema debe manejar errores sin depender de modificaciones manuales del código durante la demostración.

24. Proyecto 22: Plataforma inteligente de gestión de compras empresariales

Enfoque de IA

Forecasting + recomendación

24.1 Descripción y objetivo

Gestionar proveedores, solicitudes, órdenes y consumos y sugerir cantidades de reposición basadas en demanda histórica y reglas de negocio. La propuesta debe desarrollarse como un sistema de software completo. El componente inteligente debe integrarse mediante interfaces bien definidas, registrar su versión y poder evaluarse independientemente de la interfaz de usuario.

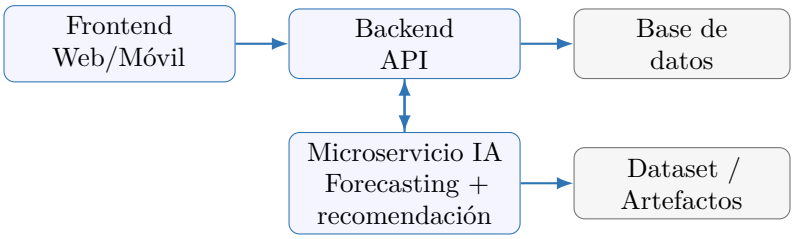
24.2 Usuarios, alcance y requisitos sugeridos

Actores sugeridos: usuario final, operador especializado y administrador. Módulos mínimos:

- Proveedores
- Solicitudes
- Órdenes
- Recepciones
- Forecast
- Recomendaciones

Como requisitos no funcionales se deben considerar seguridad, validación, disponibilidad, manejo de errores, auditoría, rendimiento, mantenibilidad, accesibilidad, protección de datos, pruebas y documentación.

24.3 Arquitectura propuesta



El backend concentra reglas de negocio y autorización; el servicio de IA recibe entradas validadas y devuelve predicciones con metadatos. Esta separación facilita pruebas, versionado, observabilidad y reemplazo del modelo.

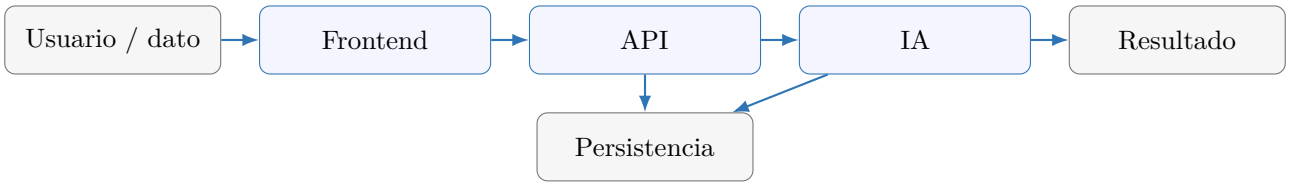
24.4 Tecnologías sugeridas

Python, FastAPI, PostgreSQL, React, scikit-learn. El equipo puede sustituirlas justificando equivalencia funcional y arquitectónica.

24.5 Paso a paso de implementación

1. **Problema y visión.** Delimitar necesidad, usuarios, objetivos, alcance, exclusiones y métricas de éxito.
2. **Ingeniería de requisitos.** Crear épicas e historias de usuario, criterios de aceptación, requisitos no funcionales y matriz de trazabilidad.
3. **Análisis y dominio.** Identificar entidades, reglas, eventos, estados, excepciones y responsabilidades de los actores.
4. **Prototipado.** Diseñar wireframes del flujo principal y validarlos antes de programar.
5. **Arquitectura.** Crear diagramas de contexto, contenedores/componentes, secuencia y despliegue; especificar endpoints.
6. **Diseño de datos.** Definir modelo lógico, restricciones, índices, migraciones y política de auditoría.
7. **Dataset.** Documentar procedencia, licencia, variables, distribución, calidad, faltantes, sesgos y criterios de partición.
8. **EDA y baseline.** Explorar los datos y construir un modelo base sencillo que establezca una referencia cuantitativa.
9. **Entrenamiento.** Construir pipeline reproducible, seleccionar hiperparámetros usando validación y conservar un test independiente.
10. **Evaluación IA.** Medir desempeño global y por subgrupos pertinentes, analizar errores y documentar limitaciones.
11. **Backend.** Implementar lógica, persistencia, seguridad, validación, manejo de excepciones y documentación OpenAPI.
12. **Servicio IA.** Exponer inferencia mediante interfaz desacoplada y registrar versión, fecha y parámetros necesarios para trazabilidad.
13. **Frontend.** Implementar formularios, dashboards, estados de error/carga y explicación apropiada de resultados.
14. **Integración.** Ejecutar casos de extremo a extremo y verificar consistencia entre UI, API, datos y modelo.
15. **Testing.** Implementar pruebas unitarias, integración, contrato/API, sistema, aceptación y pruebas específicas para datos/modelo.
16. **DevOps/MLOps.** Configurar repositorio, CI, dependencias, variables de entorno, Docker, logging y versionado de modelos.
17. **Resultados.** Reportar MAE de demanda; rotación; faltantes; exceso de inventario; tiempo del ciclo de compra. Comparar baseline y modelo final e interpretar los errores.
18. **Seguridad y uso responsable.** Crear análisis de amenazas, privacidad, sesgo, abuso, supervisión humana y recuperación ante fallos.
19. **Despliegue.** Preparar ambiente reproducible, datos demo, health checks y procedimiento de instalación.
20. **Cierre.** Elaborar informe, manuales, presentación, demostración y retrospectiva técnica.

24.6 Diagrama del flujo de extremo a extremo



24.7 Plan de pruebas

El equipo debe diseñar pruebas unitarias para servicios y reglas; integración para persistencia y servicio IA; pruebas de API; casos de aceptación derivados de historias de usuario; pruebas de entradas inválidas; evaluación del modelo con test independiente; y una prueba básica de rendimiento. Los defectos encontrados deben registrarse y relacionarse con la corrección aplicada.

24.8 Entregables específicos

Visión y alcance, SRS/backlog, matriz de trazabilidad, prototipos, diagramas TikZ/UML/C4, modelo de datos, repositorio Git, aplicación ejecutable, servicio IA, dataset o instrucciones reproducibles, modelo versionado, pruebas, documentación OpenAPI, Docker, manual técnico, manual de usuario, informe de métricas y video/demo.

24.9 Criterios de aceptación académica

El flujo principal debe funcionar de extremo a extremo y ser demostrable con datos de prueba. La IA debe estar realmente integrada y comparada contra un baseline. Los resultados deben ser reproducibles; los requisitos deben vincularse con pruebas; y el sistema debe manejar errores sin depender de modificaciones manuales del código durante la demostración.

25. Proyecto 23: Sistema de detección de defectos en documentos digitalizados

Enfoque de IA

Visión artificial + OCR quality assessment

25.1 Descripción y objetivo

Evaluar calidad de documentos escaneados detectando desenfoque, inclinación, recortes o bajo contraste antes de archivarlos. La propuesta debe desarrollarse como un sistema de software completo. El componente inteligente debe integrarse mediante interfaces bien definidas, registrar su versión y poder evaluarse independientemente de la interfaz de usuario.

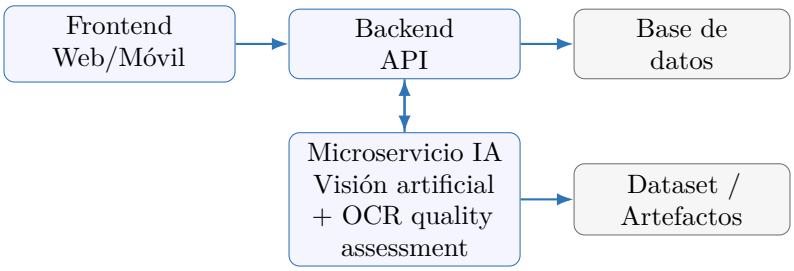
25.2 Usuarios, alcance y requisitos sugeridos

Actores sugeridos: usuario final, operador especializado y administrador. Módulos mínimos:

- Carga
- Control de calidad
- Detección de defectos
- Corrección opcional
- Aprobación
- Repositorio

Como requisitos no funcionales se deben considerar seguridad, validación, disponibilidad, manejo de errores, auditoría, rendimiento, mantenibilidad, accesibilidad, protección de datos, pruebas y documentación.

25.3 Arquitectura propuesta



El backend concentra reglas de negocio y autorización; el servicio de IA recibe entradas validadas y devuelve predicciones con metadatos. Esta separación facilita pruebas, versionado, observabilidad y reemplazo del modelo.

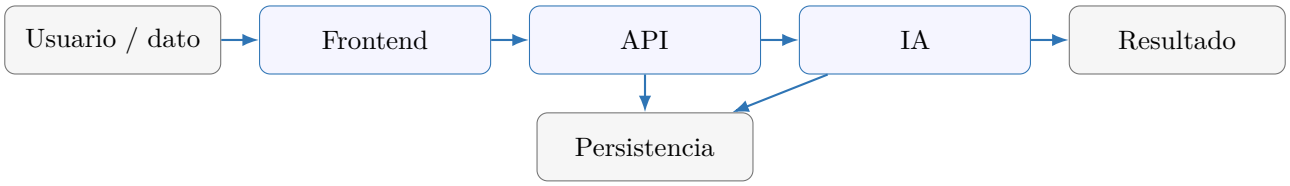
25.4 Tecnologías sugeridas

Python, OpenCV, FastAPI, PostgreSQL, React, PyTorch opcional. El equipo puede sustituirlas justificando equivalencia funcional y arquitectónica.

25.5 Paso a paso de implementación

1. **Problema y visión.** Delimitar necesidad, usuarios, objetivos, alcance, exclusiones y métricas de éxito.
2. **Ingeniería de requisitos.** Crear épicas e historias de usuario, criterios de aceptación, requisitos no funcionales y matriz de trazabilidad.
3. **Análisis y dominio.** Identificar entidades, reglas, eventos, estados, excepciones y responsabilidades de los actores.
4. **Prototipado.** Diseñar wireframes del flujo principal y validarlos antes de programar.
5. **Arquitectura.** Crear diagramas de contexto, contenedores/componentes, secuencia y despliegue; especificar endpoints.
6. **Diseño de datos.** Definir modelo lógico, restricciones, índices, migraciones y política de auditoría.
7. **Dataset.** Documentar procedencia, licencia, variables, distribución, calidad, faltantes, sesgos y criterios de partición.
8. **EDA y baseline.** Explorar los datos y construir un modelo base sencillo que establezca una referencia cuantitativa.
9. **Entrenamiento.** Construir pipeline reproducible, seleccionar hiperparámetros usando validación y conservar un test independiente.
10. **Evaluación IA.** Medir desempeño global y por subgrupos pertinentes, analizar errores y documentar limitaciones.
11. **Backend.** Implementar lógica, persistencia, seguridad, validación, manejo de excepciones y documentación OpenAPI.
12. **Servicio IA.** Exponer inferencia mediante interfaz desacoplada y registrar versión, fecha y parámetros necesarios para trazabilidad.
13. **Frontend.** Implementar formularios, dashboards, estados de error/carga y explicación apropiada de resultados.
14. **Integración.** Ejecutar casos de extremo a extremo y verificar consistencia entre UI, API, datos y modelo.
15. **Testing.** Implementar pruebas unitarias, integración, contrato/API, sistema, aceptación y pruebas específicas para datos/modelo.
16. **DevOps/MLOps.** Configurar repositorio, CI, dependencias, variables de entorno, Docker, logging y versionado de modelos.
17. **Resultados.** Reportar F1 por defecto; tasa de rechazo correcto; tiempo de procesamiento; legibilidad/OCR antes y después. Comparar baseline y modelo final e interpretar los errores.
18. **Seguridad y uso responsable.** Crear análisis de amenazas, privacidad, sesgo, abuso, supervisión humana y recuperación ante fallos.
19. **Despliegue.** Preparar ambiente reproducible, datos demo, health checks y procedimiento de instalación.
20. **Cierre.** Elaborar informe, manuales, presentación, demostración y retrospectiva técnica.

25.6 Diagrama del flujo de extremo a extremo



25.7 Plan de pruebas

El equipo debe diseñar pruebas unitarias para servicios y reglas; integración para persistencia y servicio IA; pruebas de API; casos de aceptación derivados de historias de usuario; pruebas de entradas inválidas; evaluación del modelo con test independiente; y una prueba básica de rendimiento. Los defectos encontrados deben registrarse y relacionarse con la corrección aplicada.

25.8 Entregables específicos

Visión y alcance, SRS/backlog, matriz de trazabilidad, prototipos, diagramas TikZ/UML/C4, modelo de datos, repositorio Git, aplicación ejecutable, servicio IA, dataset o instrucciones reproducibles, modelo versionado, pruebas, documentación OpenAPI, Docker, manual técnico, manual de usuario, informe de métricas y video/demo.

25.9 Criterios de aceptación académica

El flujo principal debe funcionar de extremo a extremo y ser demostrable con datos de prueba. La IA debe estar realmente integrada y comparada contra un baseline. Los resultados deben ser reproducibles; los requisitos deben vincularse con pruebas; y el sistema debe manejar errores sin depender de modificaciones manuales del código durante la demostración.

26. Proyecto 24: Plataforma de gestión de servicios de campo con predicción de duración

Enfoque de IA

Regresión + planificación

26.1 Descripción y objetivo

Administrar órdenes de servicio, técnicos, ubicaciones y repuestos y estimar duración de cada visita para apoyar la programación. La propuesta debe desarrollarse como un sistema de software completo. El componente inteligente debe integrarse mediante interfaces bien definidas, registrar su versión y poder evaluarse independientemente de la interfaz de usuario.

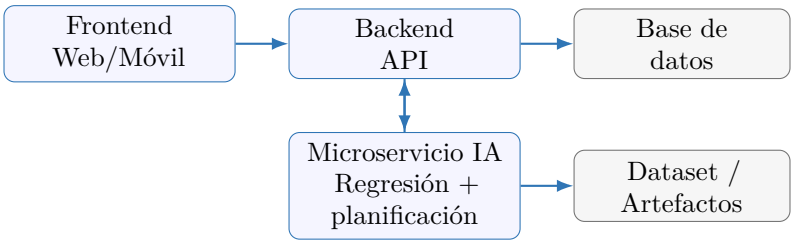
26.2 Usuarios, alcance y requisitos sugeridos

Actores sugeridos: usuario final, operador especializado y administrador. Módulos mínimos:

- Clientes
- Órdenes
- Técnicos
- Repuestos
- Predicción de duración
- Agenda

Como requisitos no funcionales se deben considerar seguridad, validación, disponibilidad, manejo de errores, auditoría, rendimiento, mantenibilidad, accesibilidad, protección de datos, pruebas y documentación.

26.3 Arquitectura propuesta



El backend concentra reglas de negocio y autorización; el servicio de IA recibe entradas validadas y devuelve predicciones con metadatos. Esta separación facilita pruebas, versionado, observabilidad y reemplazo del modelo.

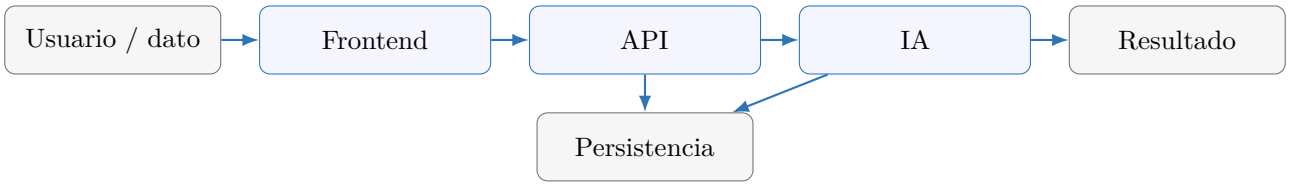
26.4 Tecnologías sugeridas

Python, FastAPI, PostgreSQL/PostGIS, React, scikit-learn, OR-Tools. El equipo puede sustituirlas justificando equivalencia funcional y arquitectónica.

26.5 Paso a paso de implementación

1. **Problema y visión.** Delimitar necesidad, usuarios, objetivos, alcance, exclusiones y métricas de éxito.
2. **Ingeniería de requisitos.** Crear épicas e historias de usuario, criterios de aceptación, requisitos no funcionales y matriz de trazabilidad.
3. **Análisis y dominio.** Identificar entidades, reglas, eventos, estados, excepciones y responsabilidades de los actores.
4. **Prototipado.** Diseñar wireframes del flujo principal y validarlos antes de programar.
5. **Arquitectura.** Crear diagramas de contexto, contenedores/componentes, secuencia y despliegue; especificar endpoints.
6. **Diseño de datos.** Definir modelo lógico, restricciones, índices, migraciones y política de auditoría.
7. **Dataset.** Documentar procedencia, licencia, variables, distribución, calidad, faltantes, sesgos y criterios de partición.
8. **EDA y baseline.** Explorar los datos y construir un modelo base sencillo que establezca una referencia cuantitativa.
9. **Entrenamiento.** Construir pipeline reproducible, seleccionar hiperparámetros usando validación y conservar un test independiente.
10. **Evaluación IA.** Medir desempeño global y por subgrupos pertinentes, analizar errores y documentar limitaciones.
11. **Backend.** Implementar lógica, persistencia, seguridad, validación, manejo de excepciones y documentación OpenAPI.
12. **Servicio IA.** Exponer inferencia mediante interfaz desacoplada y registrar versión, fecha y parámetros necesarios para trazabilidad.
13. **Frontend.** Implementar formularios, dashboards, estados de error/carga y explicación apropiada de resultados.
14. **Integración.** Ejecutar casos de extremo a extremo y verificar consistencia entre UI, API, datos y modelo.
15. **Testing.** Implementar pruebas unitarias, integración, contrato/API, sistema, aceptación y pruebas específicas para datos/modelo.
16. **DevOps/MLOps.** Configurar repositorio, CI, dependencias, variables de entorno, Docker, logging y versionado de modelos.
17. **Resultados.** Reportar MAE de duración; puntualidad; utilización de técnicos; reprogramaciones. Comparar baseline y modelo final e interpretar los errores.
18. **Seguridad y uso responsable.** Crear análisis de amenazas, privacidad, sesgo, abuso, supervisión humana y recuperación ante fallos.
19. **Despliegue.** Preparar ambiente reproducible, datos demo, health checks y procedimiento de instalación.
20. **Cierre.** Elaborar informe, manuales, presentación, demostración y retrospectiva técnica.

26.6 Diagrama del flujo de extremo a extremo



26.7 Plan de pruebas

El equipo debe diseñar pruebas unitarias para servicios y reglas; integración para persistencia y servicio IA; pruebas de API; casos de aceptación derivados de historias de usuario; pruebas de entradas inválidas; evaluación del modelo con test independiente; y una prueba básica de rendimiento. Los defectos encontrados deben registrarse y relacionarse con la corrección aplicada.

26.8 Entregables específicos

Visión y alcance, SRS/backlog, matriz de trazabilidad, prototipos, diagramas TikZ/UML/C4, modelo de datos, repositorio Git, aplicación ejecutable, servicio IA, dataset o instrucciones reproducibles, modelo versionado, pruebas, documentación OpenAPI, Docker, manual técnico, manual de usuario, informe de métricas y video/demo.

26.9 Criterios de aceptación académica

El flujo principal debe funcionar de extremo a extremo y ser demostrable con datos de prueba. La IA debe estar realmente integrada y comparada contra un baseline. Los resultados deben ser reproducibles; los requisitos deben vincularse con pruebas; y el sistema debe manejar errores sin depender de modificaciones manuales del código durante la demostración.

27. Cronograma sugerido de 16 semanas

Semana	Actividad
1	Formación de equipos, selección del problema y visión.
2	Stakeholders, alcance, historias de usuario y criterios de aceptación.
3	Requisitos no funcionales, riesgos y prototipo UI/UX.
4	Arquitectura, modelo de datos y contratos API.
5	Adquisición, exploración y gobierno de datos.
6	Baseline de IA y primera evaluación.
7	Iteración del modelo y análisis de errores.
8	Backend y persistencia.
9	Frontend e integración inicial.
10	Integración completa del servicio de IA.
11	Pruebas unitarias e integración.
12	Pruebas de sistema, seguridad y rendimiento básico.
13	Docker/CI y despliegue de preproducción.
14	Evaluación con usuarios o pruebas de aceptación.
15	Correcciones, documentación y análisis de resultados.
16	Demostración, sustentación y retrospectiva.

28. Rúbrica sugerida

Criterio	Peso	Evidencia
Requisitos y análisis	15 %	Backlog, criterios, alcance, trazabilidad.
Arquitectura y diseño	15 %	Diagramas, datos, API, decisiones.
Implementación de software	20 %	Backend, frontend, persistencia, calidad.
Componente de IA	20 %	Datos, baseline, modelo, métricas, errores.
Pruebas y calidad	15 %	Unitarias, integración, aceptación, CI.
DevOps, seguridad y reproducibilidad	10 %	Docker, configuración, seguridad, despliegue.
Documentación y sustentación	5 %	README, manuales, informe y demo.

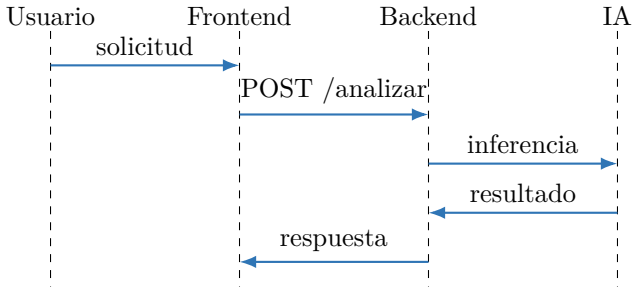
29. Estructura recomendada del repositorio

```
proyecto/
|-- backend/
|-- frontend/
|-- ai_service/
|   |-- notebooks/
|   |-- src/
|   |-- models/
```

```
|-- data/
|-- tests/
|-- docs/
|   |-- diagrams/
|-- docker-compose.yml
|-- .env.example
|-- requirements.txt / pyproject.toml
|-- README.md
```

30. Diagramas que cada equipo debe incluir

Como mínimo: diagrama de contexto, diagrama de componentes/contenedores, modelo de datos, secuencia del caso principal y diagrama del pipeline de IA. Un ejemplo de secuencia conceptual es:



31. Recomendaciones metodológicas

Se recomienda exigir que los estudiantes comparen al menos un baseline contra el modelo final, separen claramente métricas de software de métricas de IA y documenten limitaciones. En proyectos con modelos generativos, una respuesta plausible no equivale a una respuesta correcta: deben diseñarse evaluaciones, conjuntos de prueba y mecanismos de trazabilidad. En sistemas de alto impacto, la IA debe presentarse como apoyo y no como sustituto automático de decisiones humanas.

32. Bibliografía base

Referencias

[1] R. S. Pressman y B. R. Maxim, *Software Engineering: A Practitioner's Approach*, McGraw-Hill.

[2] I. Sommerville, *Software Engineering*, Pearson.

[3] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, O'Reilly.

[4] E. Gamma et al., *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley.

[5] C. Treveil et al., *Introducing MLOps*, O'Reilly.

[6] NIST, *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*, 2023.